1. Evolución del diseño web

1.1. La evolución del diseño de páginas web.

La evolución del diseño web a lo largo de estos años ha sido muy interesante.
En 1993, el National Centre for Supercomputing desarrolló el primer navegador libre llamado "Mosaic". Al año ya había unos dos millones de usuarios de este programa. Este navegador era capaz de mostrar texto y gráficos, pero tenía muchas limitaciones. No obstante, era suficiente para las páginas web de aquella fecha, las cuales eran muy lineales y funcionales para los científicos que querían compartir información. Poco tiempo más tarde aparecieron Internet Explorer y Netscape Navigator. La tecnología de estos primeros navegadores limitaba la capacidad de mostrar información gráfica necesaria para la comunicación visual. Ésta y otras restricciones influenciaron mucho al diseño de páginas web, que tenía que vérselas con problemas como las conexiones lentas con módem, los monitores monocromo y la incapacidad de los proveedores de Internet de transferir datos rápidamente.
Las páginas web de los primeros tiempos de vida de Internet eran tan simples como un documento de texto o un simple folleto en el que el desarrollo más complicado consistía en usar efectos en las fuentes y colores. El diseño solía realizarse de arriba a abajo y de izquierda a derecha, intercalando secuencias
de imágenes y textos, con muchos saltos de línea y otros elementos separadores como las barras horizontales y las viñetas de lista. Se construían en lenguaje HTML. No obstante y aunque este tipo de páginas generalmente funcionaba bien, los programadores se encontraron con problemas como la
superposición de colores o efectos de textos demasiado resaltados y sobredimensionados.

Para hacernos una idea, en la siguiente URL se encuentra una herramienta con la cual se puede observar el aspecto de cualquier página web (en caso de estar disponibles) desde su nacimiento, es decir, las diferentes actualizaciones que haya sufrido. Tan sólo hay que introducir la dirección de la página en el campo de texto:
Con el paso del tiempo, el desarrollo del comercio electrónico y las necesidades comunicativas, las páginas web comenzaron a tener estructuras más complejas y otros lenguajes de programación y metodologías entraron en juego, como **JavaScript**, que permitía crear pequeños programas muy útiles en la web. La gente empezó a darse cuenta del poder de la world wide web y de los sitios, surgiendo así el "la burbuja de Internet" acompañada del convencimiento de los desarrolladores de que había que cambiar la apariencia de las páginas por completo, hacerlas visualmente convincentes con el fin de que fueran más atractivas a los ojos del usuario.
En 1994 fue creado el consorcio W3C para establecer objetivos y estándares en el desarrollo del lenguaje HTML. Desde entonces, este lenguaje ha tenido varias versiones hasta la actual 5.0. Desde la primera, W3C y Nestcape han trabajado para alcanzar mejores prestaciones capaces de ofrecer contenido dinámico a través de la red.

Por aquel entonces surgió una nueva generación de páginas web cuyas principales características de diseño eran los iconos que reemplazaban a palabras, fondos de pantalla que se formaban a partir de mosaicos, botones con bisel y relieve, banners que sustituían a las cabeceras y el uso de desplegables. Por otra parte, estos diseños tendían a ser muy recargados con tecnologías que no respetaban el objetivo de la página. Introducían muchos iconos llamativos, gifs animados y colores primarios. Pero aún así, había diseñadores que conseguían utilizar el código HTML de maneras innovadoras consiguiendo un efecto más deseado. Así surgió la estructura de tablas cuando introdujeron la etiqueta < table >, más pensada para
introducir datos estadísticos que para el propio diseño. Los desarrolladores empezaron a utilizar

el código de las tablas para colocar en sus columnas el texto y los gráficos tal y como lo hacen los maquetadores de las revistas. De esta manera se conseguía un mejor control del posicionamiento de los elementos y se solucionaban muchos problemas estéticos.
Estas páginas aún se estaban adaptando a una tecnología constantemente cambiante, como por ejemplo, el paso de los monitores de 640x480 píxeles a los de 800x600, lo cual era una pesadilla para los diseñadores, que además, necesitaban saber si los monitores de los usuarios eran de 8 o de 24 bits para tratar la profundidad de color de los gráficos.
Otro problema era que Microsoft y Netscapte tenían cada uno sus propias reglas en sus navegadores, con lo que una página web no se veía igual en Internet Explorer que en Netscape Navigator.
Durante estos años, Microsoft y Nestcape han estado compitiendo para quedarse con el dominio del mercado de navegadores y de tecnología web, pero gracias a esta competencia han surgido muchos avances en la tecnología web.

Más tarde llegó un período en el que los desarrolladores habían aprendido de todos estos errores del pasado y comenzaron a realizar diseños agradables que permitían al visitante focalizar su atención en los principales productos y servicios que ofertaban las páginas. Los websites llegaron a convertirse en el “punto de información” de los negocios o de las empresas. Pero en este momento, los usuarios ya estaban alcanzando un nivel alto de su curva de experiencia de navegación en el que empezaron a detectar las páginas de mala calidad que simplemente servían para mostrar información de poca utilidad.
Observando este problema, la mayoría de los desarrolladores se adentraron en la interactividad, el lenguaje DHTML (HTML dinámico) y los clips Flash para impresionar a los visitantes. Estructurar los distintos elementos de la página utilizando tablas no estaba mal, pero era un método muy limitado, así que tenían que solventarlo escribiendo gran cantidad de código e implementando estilos, lo cual era un trabajo duro y complicado. Para deshacerse de este problema, introdujeron el lenguaje CSS (hojas de estilo en cascada), el cual se hizo muy popular en poco tiempo entre los desarrolladores. Lo más importante de este lenguaje es que no está basado en tablas, sino que introduce la etiqueta < div >, para asignar los estilos.
Estas páginas tenían como objetivo atraer a la audiencia y la comunicación con ella en relación con el propósito del sitio. Este propósito podría ser informar, recabar información o vender en el espacio de tiempo más corto posible. Para ello, había que dar mayor importancia al diseño del proyecto para crear una página que hiciera que el usuario pudiera ser consciente instantáneamente de a quién pertenece la página, qué ofrece, dónde encontrarlo y por qué el sitio existe. El usuario podía saber qué se ofrecía en la página y además cómo conseguirlo usando una navegación intuitiva.

Esta filosofía tendía a ser similar a la de las estrategias de marketing, es decir, la atracción de audiencia en una fracción de segundo, la retención de la atención, la inducción al deseo de informarse y la llamada a la acción. Los especialistas en marketing empezaron a “subirse al carro”, con lo cual se generó una enorme oferta de servicios de campañas publicitarias en la red. El diseño de una web de esta generación solía tener una página “splash” o una “intro” en flash para atraer la atención, una página “túnel” para guiar al usuario hacia una única oferta y una página “núcleo” que proveía de información sobre el total del contenido del sitio, además de un sistema de navegación que mostraba cómo obtener qué y dónde (en esta sección era donde se incitaba al deseo de conocer más). También solían tener una página de salida que fomentaba la llamada a la acción, como un formulario electrónico que había que rellenar para participar en una oferta.
En este punto los programadores habían alcanzado un nivel de madurez en el uso de la tecnología muy altos. Aquí surgieron la optimización de las páginas para ser encontradas y posicionadas por los motores de búsqueda, y los layouts, la última tendencia en el campo del diseño web de esos días. Los layouts permiten estrucuturar los elementos de la página y se utilizan para dirigir la

atención del usuario al principal contenido de ésta, no del diseño. Motivados por esta nueva técnica, empezó a mantenerse el fondo (background) con un diseño muy simple y a dejar la parte exterior del layout en blanco para evitar la distracción.

Una vez que se fue estandarizando la comunicación entre las distintas plataformas (PC, Mainframe, Mac, etc.), también empezaron a proliferar lenguajes de programación avanzados que permitían prestaciones de las que hasta entonces no disponían las páginas web. Se trata de PHP, C# y la tecnología ASP entre otros. Esto permitía acciones sumamente importantes como conectar la página con una base de datos, algo primordial en las tiendas online para crear catálogos de productos, por ejemplo.
El diseño web comenzó a centrarse en la integración de los elementos multimedia como el sonido, los clips de video, la animación, las bases de datos, los mundos 3D y el vrml (virtual reality modeling language). Pero los diseñadores eran conscientes de que el principal obstáculo de la utilización de estos elementos era la velocidad de descarga. Los navegadores, principalmente Netscape y Microsoft Explorer, ya estaban incrementando su tecnología en relación con el contenido multimedia debido a la demanda de éste.
El objetivo del diseño web de esta generación consistía en crear un sistema de navegación que permitiera a los usuarios encontrar lo que estaban buscando lo más rápido posible, lo cual se hace muy necesario cuando el sitio contiene grandes cantidades de información y de contenidos. El usuario debía, por ejemplo, poder reconocer para qué sirvía un formulario visualizando simplemente el botón de envío del mismo. Es por tanto que los diseñadores debieron darse cuenta de que tenían la responsabilidad de presentar el contenido y navegación necesarios para el propósito del sitio. Este trabajo era similar a crear una imagen corporativa a imprimir; qué color y qué símbolos son los apropiados para comunicar visualmente la misión o aquello que el cliente esté mirando sea capaz de lograrlo.

En definitiva, las características de esta generación de páginas web eran:

- El uso apropiado del lenguaje HTML utilizando solamente sus etiquetas estructurales en el documento y el uso de CSS.
- Respeto a la usabilidad.
- Motores de búsqueda dentro de la página para encontrar contenidos de la misma.
- Facilidad de actualización.
- Automaticidad a la hora de modificar el aspecto de la página enlazando el código a archivos CSS.
- Seguimiento de los estándares.

Durante el primer lustro de este siglo, las oportunidades surgían en la red fácilmente para aquellos que supieron reconocerlas. La tecnología ya permitía desarrollar estrategias innovadoras que, aunque en principio podían parecer ideas triviales, resultaron ser la lanzadera de muchas compañías que supieron cómo utilizarlas. Sin embargo, el cambio más notable que experimentaron las páginas web de principios de siglo no se refiere expresamente a la tecnología, aunque esté íntimamente relacionado con ella.

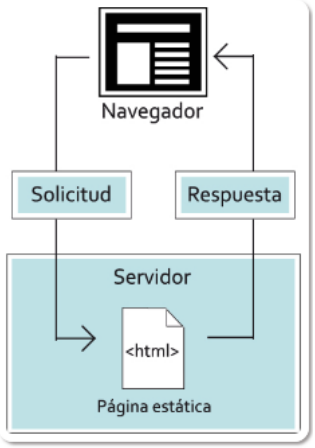
De repente y gracias al desarrollo de ciertas herramientas, los usuarios se dieron cuenta de que podían hacer muchas más cosas en Internet que simplemente mirar las páginas, también podían participar. Empezaron a aparecer los blogs, las redes sociales, los sistemas de mensajería cada vez eran más sofisticados y cualquier usuario sin mucha experiencia en informática ya podía subir su propia página web. Nació así la web 2.0., cuyos patrones de diseño se caracterizan porque el usuario puede manejar los aspectos visuales y las metáforas de presentación con tecnologías como **AJAX** y con un conjunto de lenguajes y técnicas de programación que no eran novedosos

pero que, al ser combinados, facilitan el
desarrollo de interfaces ricas y adaptables. El diseño de las páginas se vuelve elástico.
En el futuro, el diseñador profesional de páginas web no tendrá que preocuparse tanto por el aspecto de la página como por las prestaciones que ésta ofrezca. Al igual que ocurrió con el diseño industrial, la publicidad, la fabricación de coches,el video VHS y muchas otras más, poco a poco se tenderá a hacer las cosas más sencillas, más fáciles de usar y más adaptadas a las necesidades del usuario final. Nos adentramos en una era en la que la tecnología, incluidas las soluciones interactivas como las páginas web, se va a adaptar al usuario y no al revés. Esto supone un cambio fundamental en la metodología de diseño utilizada. En el sector se llama "**User-Centered Design (UCD)**" o diseño centrado en el usuario, que consiste en conducir todos los esfuerzos a satisfacer los intereses del destinatario final al que va dirigido el producto del esfuerzo creativo.
Es preciso considerar que vivimos en una era en la que la información es poder y está cada vez más en manos del usuario.

1.2. Diferencia entre una página estática y una dinámica.

Definición: Una página web es una fuente de información adaptada para la World Wide Web que se visualiza a través de un navegador de Internet o cliente web. Está compuesta por multitud de archivos que pueden ser de imagen, de video y por supuesto los archivos del código fuente. Éste debe estar escrito como mínimo en lenguaje HTML y debe indicar en su código dónde colocar cada elemento, texto, imagen o video, es decir, la estructura de la página.
En la web se pueden crear dos tipos de páginas: estáticas y dinámicas.

- Las **páginas estáticas** son aquellas que se construyen con lenguaje HTML, pueden presentar textos estáticos acompañados de imágenes o archivos multimedia (sonido o video) y a parte de los enlaces y el envío de formularios, no permiten interactividad con el usuario. Son fáciles de crear, aunque ofrecen pocas ventajas a los visitantes.
  Un sitio web estático está formado por un conjunto de páginas y de archivos HTML relacionados entre sí y que están alojados en un servidor web.
  El servidor web suministra las páginas web como respuesta a las peticiones de los navegadores, las cuales se generan cuando el usuario hace clic en un enlace, elige un marcador en un navegador o introduce una URL en la barra de direcciones del navegador. El contenido final de una página web estática lo decide el diseñador y éste no varía cuando se solicita la página. El diseñador escribe las líneas de código HTML de la página antes de subirla al servidor. El código HTML no cambia una vez colocado en el servidor y por ello, este tipo de páginas se denomina página estática.
  1. El usuario solicita la página web al servidor a través del navegador.
  2. El servidor localiza la página en sus unidades de almacenamiento.
  3. El servidor envía la página al navegador del usuario que la solicitó.



- Las **páginas dinámicas** contienen elementos que permiten al usuario interactuar con ellas, o bien, presentan efectos especiales. Para crearlas, aparte de HTML, se necesitará otro tipo de lenguajes que se describirán más adelante. Sin embargo no hay que olvidar la importancia del HTML, ya que éste es la base del desarrollo web. Cuando se construye una página dinámica, el código correspondiente a los demás lenguajes (código que permite crear los efectos especiales o interactividad) de programación se incluye embebido dentro del HTML y se suele denominar "script", aunque no todo programa interpretado es considerado un script.
  Las páginas dinámicas pueden ejecutarse en el cliente (navegador) o en el servidor.
  Cuando una página HTML contiene script de cliente, el navegador se encarga de interpretarlo y ejecutarlo. Que se ejecuten en uno u otro dependerá del uso que se le vaya a dar a la página. Por ejemplo, usos típicos de las páginas de cliente serían: efectos especiales para webs como rollovers o control de ventanas, cálculos, presentaciones en las que el texto se desplace por la página, etc. Las páginas dinámicas de cliente se escriben básicamente en dos lenguajes de programación: Javascript y VBScript, que se verán más adelante.

Desventajas:

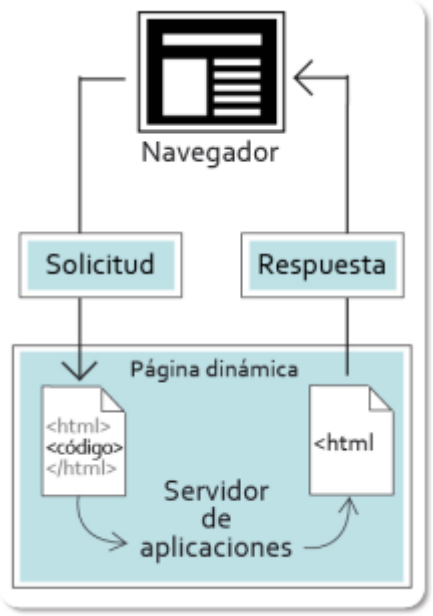
Este tipo de páginas tienen la desventaja de que son muy dependientes del navegador en el que se ejecutan y teniendo en cuenta que cada uno tiene sus propias características, no todas las páginas se visualizarán igual en todos los navegadores.

Ventajas:

Como ventajas se pueden citar que al ejecutarse en el navegador descargan de trabajo al servidor, que su respuesta ante la petición de los usuarios es muy rápida y que permiten la utilización de recursos de la máquina local.

Las páginas dinámicas de servidor son ejecutadas por el servidor. El archivo HTML de la página web llega al servidor y éste ejecuta los script contenidos en él devolviendo al cliente sólo código HTML. Las páginas dinámicas de servidor se usan en aplicaciones web tales como: foros, chats, ... y más concretamente en aquellas aplicaciones que requieren el acceso a una base de datos situada en el servidor, como por ejemplo un banco.

- El usuario solicita la página web dinámica al servidor a través del navegador.
- El servidor localiza la página en sus unidades de almacenamiento y la envía al servidor de aplicaciones.
- El servidor de aplicaciones busca las instrucciones en la página y construye el resultado final.
- El servidor de aplicaciones envía la página final al navegador del usuario.



Las principales tecnologías utilizadas del lado del servidor son: **ASP, JSP, CGI, PHP.**

Ventajas de las páginas dinámicas de servidor:

- El cliente no puede ver los script.
- Los script son independientes del navegador del usuario.

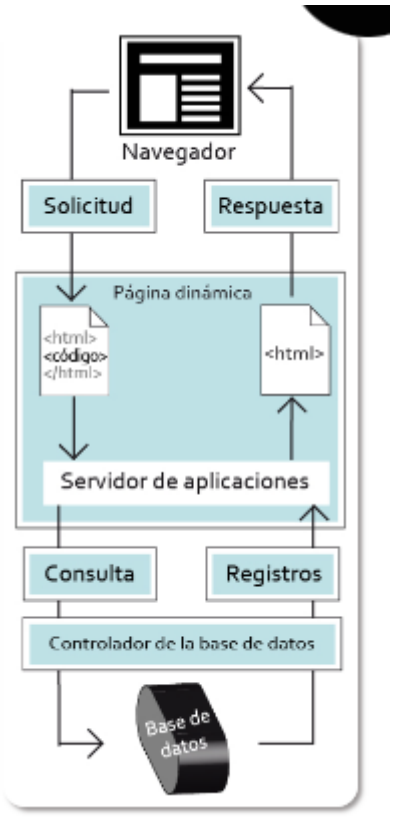
Desventajas:

- Necesidad de disponer de un servidor más potente y con más capacidades que el necesario para las
  páginas dinámicas de cliente.
- Debido al alto tiempo de procesamiento individual, estos servidores soportarán menos usuarios
  concurrentes.

1.3. Páginas web con conexión a bases de datos.

Actualmente resulta indispensable, sobre todo para las páginas web relacionadas con el comercio electrónico, conectar la interfaz con una base de datos para mostrar contenidos que se puedan someter a diferentes criterios de búsqueda, es el caso de los catálogos de las tiendas online.
Para ello es necesario que la página esté alojada en un servidor de aplicaciones que permita trabajar con recursos del lado del servidor, en este caso las bases de datos. Por ejemplo, una página dinámica puede ordenar al servidor de aplicaciones que filtre datos de una base de datos y

los inserte en el código HTML de la página.
El hecho de introducir diferentes criterios de búsqueda en una base de datos y solicitar que los muestre, es lo que se suele llamar “consulta”. Estos criterios de búsqueda están definidos en un lenguaje de programación para bases de datos denominado **SQL (Structured Query Language**, lenguaje de consulta estructurado). La consulta SQL se ejecuta en los scripts o etiquetas del lado del servidor de la página
Un servidor de aplicaciones no se puede comunicar directamente con una base de datos porque sus formatos no son compatibles. El servidor de aplicaciones sólo se puede comunicar con la base de datos por medio de un controlador que actúe de intermediario, un software que actúa como un intérprete entre el servidor de aplicaciones y la base de datos.



Cuando el controlador establece la comunicación, la consulta se ejecuta en la base de datos y se origina un conjunto de registros. Un conjunto de registros es un grupo de datos extraídos de una o varias tablas de una base de datos. El conjunto de registros se devuelve al servidor de aplicaciones, que los emplea para mostrarlos en la página.
Algunos ejemplos de consulta de base de datos sencilla escrita en SQL serían:

- Consulta para mostrar todos los campos de una tabla:

```
SELECT * FROM NOMBRE_TABLA
```

- Consulta para mostrar el valor máximo de un campo de la tabla:

```
SELECT MAX (CAMPO_VALOR) FROM TABLA_TOTALES
```

- Consulta para mostrar, por ejemplo, el nombre y el teléfono de la tabla clientes:

```
SELECT Nombre, Teléfono FROM Clientes
```

Se puede utilizar prácticamente cualquier base de datos con su aplicación web, siempre y cuando se haya instalado el controlador de base de datos correcto en el servidor.
Si el objetivo es desarrollar pequeñas aplicaciones en la página, se puede utilizar una base de datos basada en archivos como las de Microsoft Access. En cambio, si el objetivo es desarrollar aplicaciones empresariales críticas, lo aconsejable es utilizar una base de datos basada en servidor, como las que permite crear Microsoft SQL Server, Oracle Data Base o MySQL. Si la base de datos está guardada en un sistema distinto del servidor Web, hay que asegurarse de disponer de una conexión rápida entre ambos
sistemas para que la aplicación web pueda funcionar de forma rápida y eficiente.

1.4. Objetivos de las páginas web.

1.4.1. Tipos de sitios web.

Recuerda
Un sitio web o website es un conjunto de páginas web (archivos) comunes a un dominio o subdominio de Internet.

Es importante, no confundir sitio web con página web, ya que ésta última es sólo un archivo y forma parte de un sitio web. Cuando se solicita una página en el navegador insertando su dirección (URL), por ejemplo:
www.grupovertice.com, en realidad se está haciendo referencia a un sitio web, que tiene una página inicial y que es lo primero que se visualiza, pudiendo existir en ella hipervínculos al resto de páginas del sitio.

Existe una amplia gama de sitios web clasificados según su contenido o el uso para el que haya sido creado. El siguiente esquema muestra los tipos de clasificación de sitios web:

Sitios estáticos:
Los usuarios no pueden modificar el texto de la página, encargándose exclusivamente los diseñadores de su contenido.
Antes de diseñar un sitio web es muy importante encuadrar, entre los objetivos del mismo, el tipo

de website del que se va a tratar, puesto que esto nos ayudará a poner en práctica estrategias y metodologías propias de cada tipo que ayudarán a posicionar la página y a diseñar su estructura adecuadamente.

Sitios dinámicos:
Permiten interactividad con los usuarios, personalizando éstos la información que se les ofrece.

Sitios públicos:
Se tratan de sitios web dirigidos al público en general, sin restricciones de acceso en principio.

Extranet:
Este tipo de sitios limitan el tipo de usuarios que pueden acceder, por ejemplo los proveedores de una empresa o los clientes.

Intranet:
Son sitios de acceso restringido a una empresa u organización. Normalmente suelen publicarse en redes privadas.

Estructura abierta:
Los usuarios pueden acceder a cualquier página del sitio web, ya que todas ellas disponen de su dirección.

Estructura cerrada:
Son sitios en los que los usuarios no pueden acceder a todas las páginas que lo forman. Por ejemplo, las páginas que solicitan un registro para acceder a ellas.

Estructura semicerrada:
Los usuarios tienen que pasar por puntos específicos para poder acceder a las secciones más importantes.

Por su profundidad:
Este tipo de clasificación está basada en el número de enlaces que el usuario debe pulsar para llegar al contenido. Se aconseja que el número de enlaces no sea mayor de tres.

Sitios comerciales:
Tienen una finalidad económica. Los usuarios que visitan este tipo de sitios suelen ser clientes, empleados, inversores, los sectores de la competencia, etc. Los sitios comerciales se dividen a su vez en:
a) **Corporativos**: informan sobre la empresa.
b) **Promocionales**: informan sobre los productos.

Sitios informativos:
Distribuyen información a los usuarios que dependerán del tipo de contenido que ofrezcan.

Sitios de ocio:
Este tipo de sitios son un caso especial, suelen seguir reglas propias en su diseño.
Su finalidad suele ser económica.

Sitios de navegación:
Ayudan al usuario a encontrar lo que busca en internet. Dentro de este tipo de sitios se encuentran los portales, sitios web que proporcionan un punto de inicio, entrada o portal a otros recursos en Internet o una intranet.

Sitios artísticos:
Muestran la expresión artística del diseñador y las únicas normas por las que se rige son las que el propio diseñador o diseñadores dicten.

Sitios personales:
Mantenido por una persona o un pequeño grupo (como por ejemplo familia), contiene información o cualquier contenido que la persona quiera incluir. En este grupo se incluyen los **blogs**.

1.4.2. Introducción a la interfaces gráficas. Características deseables de todo sitio web.

El diseño de la interfaz de un sitio web necesita ser capaz de comunicar claramente lo que éste ofrece y cómo los usuarios pueden hacer uso de él. Diseñar websites efectivos requiere tener en cuenta consideraciones técnicas (tiempo de descarga y accesibilidad), estructurales (organización y flujo de trabajo) y visuales (personalización).
Los elementos que forman una interfaz gráfica de usuario (GUI) son objetos que se configuran a gusto del diseñador, que van a responder sobre determinados eventos que pueden ocurrir durante la ejecución de la aplicación y deben ser programados por el equipo de desarrollo. Ejemplos de estos objetos son:

- Contenedores (Panel, Form, etc.).
- Botones (Button).
- Cajas de texto (TextField).
- Casillas de verificación (CheckBox).
- Etiquetas (Label).
- Imágenes (Image).
- Listas desplegables (ComboBox).
- Etc.

Las características deseables para el diseño de una interfaz de usuario son:

- Conocer los distintos tipos de usuarios que van a utilizar la interfaz para personalizar el diseño.
- Emplear modelos de diseño bien definidos.
- Presentar la información adaptada a cada tipo de usuario.
- Establecer sistemas de interacción uniformes.
- Utilizar en cada momento lo que resulte más sencillo y apropiado.
- Utilizar los elementos necesarios y en su justa medida.
- Mantener en todo momento una estructura visual del conjunto de toda la interfaz.
- Diseñar facilitando la modularización y usabilidad de los módulos o componentes de la interfaz.
- Utilizar metáforas del mundo real para facilitar su manejo.
- Hacer un uso adecuado de imágenes e iconos de objetos.
- Utilizar guías de estilo adecuadas para cada interfaz.

Muchos diseñadores aún se inclinan hacia los métodos clásicos de desarrollo, aún cuando cada día surgen nuevos y mejores procedimientos de diseño que se ajustan a los estándares. Uno de estos estándares sería el de W3C (World Wide Web Consortium, consorcio internacional que produce recomendaciones para la World Wide Web), por el cual nos estaremos asegurando que la página podrá visualizarse en cualquier navegador, PDA o teléfono móvil que disponga de Internet.

Además, ajustarse a estándares permite la compatibilidad con nuevas generaciones de dispositivos móviles y facilita las actualizaciones y la manutención en el futuro. Por tanto, éste sería un criterio importante a seguir.
En la siguiente página web, introduciendo la URL de nuestra página web podremos saber si ésta se ajusta a los estándares W3C o no:
Ajustes estandar W3C: https://validator.w3.org/
La tendencia en el diseño futuro de interfaces gráficas, sigue cuatro líneas de actuación:

- reconocimiento de caracteres,
- síntesis de habla (Text to speech: Loquendo y Ghostreader),
- reconocimiento de habla
- procesamiento de imágenes y video.

Estas líneas proporcionarán:

- Entornos altamente virtualizados (realidad virtual).
- Máquinas que sigan los patrones deductivos y se comuniquen de una forma semejante a la del hombre. Traslado del Proceso de la información al Proceso de los conocimientos.
- Reacciones de los entornos a estados de ánimo y temperamento, peticiones explícitas, etc.

1.5. Estructura básica de una página web.

El esqueleto básico de una página web estaría formado por:

La cabecera.

Es una de las partes más importantes, ya que en ella se van a definir parámetros que ayudarán a configurar y optimizar la página (en el código). En la cabecera se apoyan bastante los buscadores, de
ahí la importancia de esta sección dentro de la página. Este elemento debería responder a la pregunta ¿de qué es esta página?
Normalmente este espacio incluye el logotipo de la empresa o el nombre (que normalmente es "pulsable" y conduce a la página principal) y, algunas veces, añade una coletilla como el lema de la empresa o la actividad. Además, es donde suelen colocarse los banners. Por ejemplo:

Menú de navegación:

Identifica las páginas del website y facilita la navegación a los clientes.

Cuerpo

Muestra el contenido de la página.

Pie de página

Se trata de un elemento opcional que suele contener información muy concreta.

Uno de los usos más comunes del pie de página es el de introducir información acerca de la empresa propietaria del sitio web o responsable directo de ella, se suele incluir el número de teléfono y fax, información sobre copyright de los contenidos, política de privacidad, etc. El pie puede contener también un menú auxiliar que permita al usuario continuar navegando por el sitio web sin tener que volver a buscar el menú principal (esto suele ocurrir en páginas muy altas). Otro uso del pie es el de contener un menú específico, en el que se suelen mostrar enlaces a servicios muy particulares del sitio web, como formulario de contacto, condiciones de uso, ofertas de empleo, etc.

1.6. Introducción a los lenguajes básicos de diseño web.

1.6.1. HTML.

Todo diseñador de páginas web debe dominar el lenguaje HTML:
Recuerda
HTML (HyperText Markup Language, Lenguaje de Marcas Hipertextuales) es un lenguaje de marcación diseñado para estructurar textos y presentarlos en forma de hipertexto, formato

estándar de las páginas web.
Gracias a Internet y a los navegadores más comunes, Internet Explorer, Netscape, Firefox, Safari, etc., HTML se ha convertido en uno de los formatos más populares y fáciles de aprender que existen para la elaboración de documentos para la web.
HTML utiliza etiquetas o marcas, que consisten en breves instrucciones de comienzo y final, mediante las cuales se determinan la forma en la que debe aparecer en su navegador el texto, las imágenes y demás elementos.

La estructura básica de un documento HTML es la siguiente:

```
<html>
<head>Es la cabecera
<title>Título de la página que aparece en la barra de títulos
</title>
</head>
<body>
Aquí van todas las etiquetas o instrucciones del documento
</body>
</html>
```

La etiqueta limita el documento e indica que está escrito en este lenguaje.

Un documento HTML se divide en dos partes:

- Cabecera
  Está comprendida por las etiquetas y . Dentro se puede encontrar información del documento que no se verá en la página web. Destaca la etiqueta . El título que se le ponga a la página debe ser breve y descriptivo de su contenido, ya que será lo que verán los usuarios cuando incluyan la página a su agenda de direcciones “bookmark“ y en la barra de título.
- Cuerpo

  Está comprendido por las etiquetas . Dentro de él se incluirá el grueso del contenido de la página web (texto, imágenes, sonidos,...).

Los documentos HTML se pueden crear con cualquier editor de texto, como por ejemplo el Bloc de notas, Gedit, GNU Emacs, etc., en el que se escribirá el código HTML con la información que debe aparecer en la página web. Una vez escrito el código en el editor de texto, guardando éste con el formato *.html se podrá abrir el archivo con un navegador y ver el resultado.
En la actualidad existen programas para la edición de código como pueden ser el famoso software de Adobe Dreamweaver o Microsoft Expression Web. A estos programas se les conoce como editores WYSIWYG (lo que ves es lo que obtienes), lo que quiere decir, que se puede visualizar el resultado en tiempo real lo que se está editando, ya que se trata de arrastrar los elementos a un espacio de trabajo.

1.6.2. Introducción a las Hojas de Estilo o CSS.

El concepto de Hojas de Estilo apareció por primera vez en 1996, cuando el estándar W3C publicó una nueva recomendación denominada “Hojas de Estilo en Cascada” o “CSS” (Cascade StyleSheet). Las Hojas de Estilo surgieron para compensar los defectos de HTML con respecto a la presentación y diseño de las páginas. Su función es la de separar el estilo lógico del estilo físico.

El estilo lógico no se preocupa de la apariencia final del documento, sólo de la estructura: cabeceras, párrafos,...

El estilo físico no tiene en cuenta la estructura del documento preocupándose por el aspecto final de éste: párrafos con un determinado tipo de letra, tablas con un cierto color de fondo, etc.
Su finalidad es la de crear unos estilos físicos separados de las etiquetas HTML (en lugar de aparecer como atributos de éstas), y aplicarlos en los bloques de texto en los que se desee cambiar el estilo. Esto facilita mucho el diseño y revisión de páginas, ya que se puede variar la presentación de una página, o de todo el conjunto de ellas, sin cambiar una sola línea del código HTML. Se denominan Hojas de Estilo en Cascada
porque se pueden definir múltiples hojas y los estilos pueden aplicarse a todas las páginas.
Los estilos se pueden aplicar tanto a una etiqueta HTML concreta, como a un conjunto de etiquetas, al documento completo o incluso se puede vincular el documento html a un archivo que contiene los estilos, por ejemplo:

```
<link rel=stylesheet type="text/css">
```

Supongamos que queremos aplicar el estilo visto anteriormente a algunas páginas.
Para ello tendría que crear un fichero de texto como el que sigue:

```
body {background: green ;font-size: 12pt; font-family:verdana;
margin-left: 0.5in; margin-right: 0.5in}
h1 {background: blue; font-size: 14pt;font-weight: bold;color: red}
h2 {font-size: 12pt;font-weight: bold; color: red}
div {background: URL(imagen.jpg)}
```

1.6.3. Introducción a los lenguajes de script.

Importante
Un lenguaje de script no está considerado como un lenguaje de programación propiamente dicho, sino como un lenguaje de órdenes, como pueden ser los lenguajes de macros que tienen muchos procesadores
de texto. El código se inserta dentro del documento HTML y se ejecuta en el navegador del usuario al cargar la página, o cuando realiza alguna acción especial como por ejemplo, pulsar sobre un enlace.

Estos lenguajes permiten variar dinámicamente el contenido del documento, validar formularios, modificar el comportamiento normal del navegador, etc. Lo que no permiten es manejar bases de datos, ya que, los scripts se ejecutan en la máquina del usuario y no en el servidor en el que se encuentra alojada la página.

El primer lenguaje de script que vió la luz fue el **JavaScript** de **Netscape**, basado en la sintaxis de Java. Su utilidad, junto a la alta cuota de mercado de la que entonces disponía Netscape, hizo que se popularizara y extendiera su uso.
El máximo rival de Netscape, **Internet Explorer de Microsoft**, introdujo el único rival serio que JavaScript ha tenido en el mercado de los lenguajes de script, el **VBScript**.
Está basado en el lenguaje BASIC y no ha tenido demasiada difusión en internet debido a la previa implantación del JavaScript y a que son de parecida funcionalidad, pero sí es utilizado dentro de Intranets y de otras aplicaciones de Microsoft como IIS, Access, Word, etc...

Un ejemplo de script para abrir un pop-up al abrir la página web HTML sería:

```
<body>
    <script language=Javascript> open('NOMBRE PAGINA.HTM','TITULO PAGINA',
    "toolbar=0,location=0,status=0,menubar=0,scrollbars=yes,resiz able=YES,
    width=490,height=250"); </script>
</body>
```

1.7. Presente y futuro de la tecnología web.

1.7.1. Presente: La Web 2.0.

En los inicios de la world wide web los entornos eran estáticos, con páginas HTML que no se actualizaban frecuentemente y no tenían interacción con el usuario, es lo que se conocía como Web 1.0. Sin embargo el
éxito de la web dependía más de las webs dinámicas, que permitían interactividad con los usuarios. Así, la Web 2.0 es la representación de la evolución de las aplicaciones tradicionales hacia aplicaciones enfocadas al usuario final.

Se puede entender como 2.0 a todas aquellas utilidades y servicios de Internet que se sustentan en una base de datos, la cual puede ser modificada por los usuarios del servicio, ya sea en su contenido (añadiendo, cambiando o borrando información o asociando datos a la información existente), bien en la forma de presentarlos, o en contenido y forma simultáneamente. (Ribes, 2007).

La web 2.0 puede ser considerada como un sistema en Internet en el que los usuarios son tan importantes como las empresas o los medios de comunicación y que está teniendo una influencia cada día mayor en la sociedad y en la economía. En las páginas web de hoy, los usuarios ya no "leen, hacen clic y nada más", sino que participan activamente comentando las noticias, anotando lo que desean recordar, participando en
foros, etc. Estamos ante un nuevo fenómeno en el que es necesario revisar las normas que conocíamos y replantearnos muchas de nuestras actividades con una mentalidad abierta.

Antes de la web 2.0 navegaban unos 45 millones de personas, era en la segunda mitad de los noventa. Estos usuarios visitaban alrededor de unas 250.000 páginas web de las que la mayoría de ellas pertenecía a aquéllos que podían permitirse el manejo de las herramientas necesarias para tener presencia en Internet; se trataba básicamente de las empresas. El dominio de lenguajes como HTML o de programas como el Dreamweaver y el poder diseñar una página estaba restringido a personas con habilidades técnicas que habitualmente realizaban dicha labor para las empresas que las tenían en su plantilla.
Así, las páginas web de finales del siglo pasado se caracterizaban por ser "webs de las empresas":

Definición
Un conjunto de contenidos organizados para conseguir que un número lo más alto posible de usuarios se congregaran a su alrededor, pudiera ver la publicidad y, preferentemente, llegaran a utilizar su tarjeta de crédito y realizar alguna compra.
Se trataba de páginas totalmente unidireccionales, no pensadas para recibir ningún tipo de retroalimentación por parte de los usuarios más allá del simple botón para efectuar una compra. Como mucho, podíamos rellenar algún formulario para solicitar información que podíamos recibir o no. Usuarios verdaderamente pasivos.

Esta nueva etapa vio nacer la creación de instrumentos de sencillo manejo y al alcance de cualquier usuario, que permitían el diseño de páginas web que parecían profesionales. La recién nacida empresa Pyra Labs desarrolló Blogger, una herramienta simple de diseño de páginas web que empezó a popularizarse a gran velocidad hasta que, en febrero de 2003 fue adquirida por Google. El desarrollo de Blogger, unido al de otras empresas de la época como SixApart o Live Journal, dio lugar a la aparición de una gran variedad de páginas web personales dotadas de una estructura característica: en modo cronológico inverso, con aspecto de diario de navegación y generalmente plagadas de enlaces que
permitían al usuario recorrer hacia atrás el camino seguido por aquél que publicaba una historia. Mediante este tipo de páginas, muchos usuarios que antes eran pasivos decidieron crear sus propios contenidos, contribuyendo así al desarrollo de una auténtica inteligencia colectiva. En esta nueva etapa, en la web ya no sólo hay muchos más usuarios, sino que además son mucho más activos.
Los blogs no son sinónimo de web 2.0, existen muchos más fenómenos relacionados con ella, pero sí probablemente sea la herramienta más llamativa y la que mejor explique el fundamento de este cambio. Además, la abundancia de contenidos procedente de los blogs y publicaciones de tipo personal posibilita muchas de las características de la web 2.0.
Las personas ya no se limitan a introducir sus contenidos sino que empiezan a desarrollar otro tipo de actividades como asignar a sus contenidos una o varias palabras clave, asociarlos entre sí por categorías, unir entradas mediante enlaces, almacenarlos en repositorios personales y compartirlos, o bien, promover su circulación en filtros sociales en los que los usuarios votan y comentan las noticias.
Las consecuencias de estas nuevas capacidades inciden prácticamente en toda la cadena de relaciones de Internet. Los usuarios son ahora ascendidos a la categoría de creadores de contenidos, han alterado sus pautas de comportamiento y, debido a ello, las empresas online han percibido cambios en sus relaciones que afectan a todos sus elementos comunicativos y productivos: un problema en un producto puede multiplicarse, la comunicación puede viralizarse, el diseño de un producto puede consensuarse con los usuarios, etc.

Algunos de los servicios 2.0 son:

Blogs.

Son sitios web periódicamente actualizados que muestran textos o artículos de uno o varios autores y cronológicamente, apareciendo primero el más reciente, donde el autor conserva siempre la libertad de dejar publicado lo que crea pertinente. Los usuarios sin experiencia pueden hacerse un blog con herramientas web muy sencillas.

Contenidos “a la carta” o Lector RSS.

Sirven para refundir contenidos a los suscriptores. Ejemplos son **Google Reader, Bloglines**, etc., que nos permiten mantener un seguimiento de nuestros blogs favoritos y un consumo más eficiente de la información.

Filtros sociales o recomendadores.

Páginas como **Menéame** o **Digg** tienen un importante papel en Internet como generadores de atención y tráfico. Los usuarios envían noticias que han visto en otros sitios, las votan o comentan y, según la atención que generan, llegan o no a ser promovidas a la portada, donde son leídas por muchísimos usuarios que las utilizan como filtro.

Redes sociales.

Servicios como **Myspace, Facebook, Hi5!**, etc., permiten establecer una red de relaciones y lanzar peticiones de contacto en los más diversos ámbitos, desde profesionales hasta personales. Pueden suponer una parte importante de la "personalidad online" de un usuario.

Photo sites.

Son páginas dedicadas al archivo y compartición de fotografías con usuarios. En Tuenti, por ejemplo, se pueden definir familiares, amigos, etiquetar las fotos, compartirlas, subirlas a blogs, etc.

Universos virtuales.

Servicios como **Second Life** y otros, en los que los usuarios se crean un "avatar" o personalidad virtual y desarrollan actividades de todo tipo. Su proyección empresarial se ha reforzado mucho con el establecimiento de infinidad de empresas que desean tener presencia o vender productos y servicios en ese entorno.

Video sites.

YouTube es el ejemplo por excelencia. En ellos los usuarios cuelgan vídeos, los comparten y comentan sus opiniones; tienen ciertas prestaciones de red social. Muchas empresas lanzan campañas de marketing viral a través de este tipo de páginas y muchas páginas, bloggers, etc., enlazan videos de Youtube en sus contenidos para que aparezcan y puedan ser visualizados en ellas.

Foros.

Aunque son uno de los medios de interacción más antiguos de Internet (como los chats), los foros tienen un gran papel como espacio de diálogos en movimiento. Algunos de ellos generan una atención y un tráfico muy llamativos. Suelen estar clasificados por temáticas y generan una gran fidelidad del usuario.

Social bookmarking.

Son páginas como la de www.del.icio.us, en las que los usuarios guardan sus sitios favoritos o páginas cuyas direcciones quieren recordar.

Wikis.

Páginas en las que varios autores pueden colaborar conjuntamente para enriquecer el contenido. La **Wikipedia**, considerada la mayor enciclopedia del mundo, es un ejemplo desarrollado mediante una
herramienta wiki: cualquier usuario puede crearse una cuenta y añadir o modificar información en sus páginas.

Aunque el desarrollo de este tipo de herramientas se ha dado sobre todo a escala individual, su utilización en la empresa está creciendo en vertientes muy interesantes. A los ya mencionados ejemplos de desarrollos de blogs corporativos o de presencia de empresas en Second Life, podemos añadir también el uso de foros o wikis para la coordinación de equipos de trabajo, de sitios de social bookmarking para compartir información común, de redes sociales para la gestión de contactos o incluso, de manera incipiente, de filtros sociales para facilitar el desarrollo de un sustrato común de información comentada.

1.7.2. El futuro de la tecnología web: La Web 3.0.

La Web 3.0 es una evolución de la Web 2.0 destinada a añadir significado a la red, de ahí que se la conozca como web semántica. Promete transformar no sólo nuestra experiencia web, sino toda nuestra cotidianidad. La idea de Web 3.0 surge por primera vez en el año 2006 de la mano de Jeffrey Zeldman,
crítico de la web 2.0.
Los contenidos publicados en páginas web son difícilmente entendibles por los ordenadores y tienen significado principalmente para las personas. Se trataría de añadir información adicional con una estructura tal que pueda ser entendida por los ordenadores quienes por medio de técnicas de inteligencia
artificial serían capaces de emular y mejorar la obtención de conocimiento hasta el momento reservada a las personas.
Un claro ejemplo de web semántica sería aquella que permitiera formular consultas del tipo: "Busco un trabajo que sea de media jornada, preferiblemente por las mañanas, en el campo de la informática y cuyo sueldo no sea inferior a 800 euros". El sistema le devolvería una serie de ofertas de trabajo con los criterios especificados, sin la necesidad de que el usuario pase horas y horas localizando ofertas en internet. Si realmente se lograra una combinación efectiva entre la inclusión de contenido semántico en las páginas web y el uso de inteligencia artificial, la Web 3.0 sería una auténtica revolución, pero las expectativas a corto/medio plazo son algo pesimistas ya que tradicionalmente los avances de esta ciencia han sido lentos y no exentos de dificultades. Hasta ahora estas tecnologías de "inteligencia semántica" se han implementado y usado a pequeña escala en compañías para conseguir una manipulación de datos más eficiente.

1. Las páginas mostraban tanto imágenes como texto aunque forma desordenada.
2. Aparición de distintos elementos gráficos: botones, banners, iconos para sustituir a algunas palabras,...
3. Páginas web enfocadas a sus objetivos: comerciales, informativos, comunidades,...
4. Grandes mejoras en los navegadores, casi sin errores de interpretación de código.
5. Auge de la TV IP o TV online.

1.8. Introducción al desarrollo web. Software de diseño y edición.

Definición
Se entiende por desarrollo web al proceso de creación y mantenimiento de páginas web, generalmente para ser parte de un sitio web disponible en la wold wide web.
Las personas que generalmente se encargan de esta tarea se les denominan desarrolladores web o "webmasters". Aunque con las tecnologías disponibles hoy en día, cualquier persona sin grandes conocimientos de informática puede construir una página web.
El proceso de creación de páginas web puede realizarse editando manualmente las etiquetas HTML sin más ayuda que la de un editor de texto plano. Esta tarea puede resultar muy tediosa y obliga al desarrollador a estar continuamente pendiente del código HTML. No obstante, existen editores especializados en manejar las etiquetas, de manera que el desarrollador se concentre en el diseño y el editor genere automáticamente el código HTML.

En la actualidad existe una amplia oferta de editores de este tipo, que generan código HTML de una manera más sencilla y potente que la de escribir directamente las etiquetas y ayudan a diseñar webs cada vez más espectaculares y funcionales. Los primeros editores web sólo permitían la edición de código HTML, sin embargo en la actualidad tienen soporte integrado para otras tecnologías como CSS, JavaScript, PHP,
JSP,...

Verdadero o falso

1. En la segunda fase se define la imagen que tendrá la página en función de su objetivo.

   Falsa

2. El diseño web clásico utiliza principalmente: flash, video streaming, texto, imágenes,...

   Falsa

3. En la primera fase de diseño web se realiza un estudio de mercado para ver las necesidades de los
   usuarios del sitio.

   Verdadera

4. El diseño industrial está orientado a profesionales que deseen introducir las últimas novedades
   tecnológicas en las páginas web.

   Falsa

5. En la última fase se llevan a cabo una serie de pruebas para detectar posibles errores.

   Verdadera

6. El diseño web avanzado es demandado principalmente por clientes del mundo audiovisual y empresas tecnológicas.

   Verdadera

Algunos de los editores más representativos que existen actualmente en el mercado son:

- **Adobe Dreamweaver:**
  Se trata de un editor del tipo WYSIWYG y está considerada como la herramienta de diseño de páginas web más avanzada que existe. El código que genera es de buena calidad y es capaz de diseñar páginas con un verdadero aspecto profesional. Soporta integración con lenguajes de servidor (por ejemplo PHP y JSP) y muy buenas capacidades para trabajar con CSS y capas, Javascript,.... No se trata de un software de código abierto (open source) por lo tanto hay que pagar por él.
- **Amaya:**
  Es un navegador y editor web desarrollado por el W3C. Se trata un software de código libre. Permite navegar por una página y editarla al mismo tiempo. Para más información se puede visitar la página:
  Página web de Amaya: https://www.w3.org/Amaya/
- **Firebug:**
  Se trata de un complemento o plugin para Firefox que añade funcionalidad de edición y depuración WYSIWYG mientras navega por la página.
  Como cabría esperar sólo está disponible para Firefox, por lo que no se puede utilizar para

depurar código para Internet Explorer u otros navegadores. Para más información se puede visitar la página web: Pluging Firebug https://getfirebug.com/

- **Aptana:**
  Tiene un muy buen asistente de código con un gran soporte de CSS y Javascript. Existe una versión de código abierto y permite depurar código para Firefox e Internet Explorer. Para más información
  visitar la página: Web Aptana https://www.axway.com/en/aptana
- **Adobe Fireworks:**
  Fireworks es una aplicación muy útil que sirve para generar, modificar y optimizar imágenes y gráficos para las páginas web en los formatos mapa de bits y vectorial. También permite diseñar efectos para las páginas como comportamientos al pasar el ratón por algún objeto, menús emergentes, etc. Otra de las opciones que permite Fireworks es la de "partir" una imagen para aligerar su descarga y poder exportar estos"pedazos de imagen" a una tabla HTML para que sean unidas de nuevo en el ordenador del visitante. Este recurso es muy usual.
- **Microsoft Expression Web:**
  Es una herramienta de diseño profesional para realizar páginas y aplicaciones web. Admite los estándares XHTML y CSS, de modo que permite a los diseñadores web crear fácilmente sitios independientes del navegador. Lo más interesante para las personas con poca experiencia es que al mismo tiempo que se escribe el código, se puede visualizar el resultado en otro panel del espacio de
  trabajo.
- **HTML Kit:**
  Ùn completo editor de lenguajes HTML, XHTML, XML, CSS, JavaScript y PHP entre otros. Permite a los programadores editar, formatear, validar, previsualizar y publicar páginas web. Los principantes pueden beneficiarse de sus prestaciones porque este programa emite avisos de error de código y ofrece sugerencias de cómo crear mejor la página.

Asocie los distintos contenidos de la página con la parte de dicha página en la que se sitúan:

Cuerpo: Se introducen los contenidos de la página y puede estar dividido en secciones en función de la información que muestre cada sección.
Pié de página: Es opcional y suele contener información muy concreta.
Menú de navegación: Identifica las páginas del websites.
Cabecera: Es una parte muy importante, los buscadores la utilizan para localizar las páginas.

Ideas clave 1/3

1. Las páginas web han evolucionado a lo largo de su historia tanto en diseño como en tecnología. Las primeras apenas tenían gráficos y se construían exclusivamente en lenguaje HTML y las actuales
   disponen de avanzadas aplicaciones desarrolladas en lenguajes de programación como CSS, PHP, PERL o JavaScript.
2. Una página web es una fuente de información adaptada para la World Wide Web que se visualiza a través de un navegador de Internet o cliente web. Está compuesta por multitud de archivos que
   pueden ser de imagen, de video y los archivos del código fuente.

3. La diferencia entre páginas estáticas y dinámicas reside en que las dinámicas permiten interactividad con el usuario y las estáticas apenas permiten pulsar en los enlaces y rellenar formularios.
4. Para conectar una base de datos con una interfaz web es necesario que la página esté alojada en un servidor de aplicaciones que permita trabajar con recursos del lado del servidor (con la base de datos) y un software que actúe como un intérprete entre el servidor de aplicaciones y la base de datos, es decir, un controlador.
5. Un sitio web o website es un conjunto de páginas web (archivos) comunes a un dominio o subdominio de Internet.
6. El diseño de la interfaz de un sitio web necesita ser capaz de comunicar claramente lo que éste ofrece y cómo los usuarios pueden hacer uso de él. Diseñar websites efectivos requiere tener en cuenta consideraciones técnicas (tiempo de descarga y accesibilidad), estructurales (organización y flujo de trabajo) y visuales (personalización).
7. W3C (World Wide Web Consortium) es un consorcio internacional que produce recomendaciones para la World Wide Web por el cual nos estaremos asegurando que la página podrá visualizarse en cualquier navegador, PDA o teléfono móvil que disponga de Internet.
8. La estructura básica de una página web consta de cabecera, menú de navegación, cuerpo y pie de página.
9. Las Hojas de Estilo o **CSS** surgieron para compensar los defectos de HTML con respecto a la presentación y diseño de las páginas. Su función es la de separar el estilo lógico del estilo físico.
10. **HTML** (HyperText Markup Language, Lenguaje de Marcas Hipertextuales) es un lenguaje de marcación diseñado para estructurar textos y presentarlos en forma de hipertexto, formato estándar de las páginas web.
11. Un lenguaje de **script** es un lenguaje de órdenes. El código se inserta dentro del documento HTML y se ejecuta en el navegador del usuario al cargar la página, o cuando realiza alguna acción especial como por ejemplo, pulsar sobre un enlace.
12. Se puede entender como **2.0** a todas aquellas utilidades y servicios de Internet que se sustentan en una base de datos, la cual puede ser modificada por los usuarios del servicio, ya sea en su contenido (añadiendo, cambiando o borrando información o asociando datos a la información existente), bien en la forma de presentarlos, o en contenido y forma simultáneamente.
13. Entre las aplicaciones para editar y diseñar páginas web se pueden mecionar Adobe Dreamweaver, Amaya, Firebug, Aptana y Microsoft Expression Web.

2. Hojas de estilo CSS

2.1. Concepto de Hojas de Estilo o CSS.

Recuerda
Las Hojas de Estilo se definen como bloques de código en lenguaje XML o XHTML que se encargan de separar dentro del documento HTML el estilo lógico del estilo físico, siendo el estilo lógico la estructura del
documento y el estilo físico su aspecto final.
Sus siglas en inglés vienen de Cascading Style Sheets (CSS).
Ya en 1990 los creadores de HTML estaban interesados en separar el estilo de la estructura del documento, pero la inmadurez de los navegadores y la falta de estandarización lo pusieron muy complicado. Mientras tanto, muchos autores se quejaban de que les resultaba un trabajo muy tedioso el tener que editar todo el código para múltiples páginas. Algo que se podía hacer fácilmente con editores de texto como Microsoft Word, no se podía hacer con HTML. Esto es así porque porque HTML posee escasos elementos de presentación, como por ejemplo, un único modelo de bordes de tablas o ningún control del espacio entre las líneas o entre los caracteres.
Los primeros desarrollos de las Hojas de Estilo tenían más que ver con los navegadores que con los diseñadores de páginas web, es decir, los desarrolladores de este lenguaje se centraron más en crear navegadores con los cuales un usuario pudiera modificar el estilo de una página web, pero esto rompía el equilibrio entre autores y usuarios. En 1999,CSS 2 se convirtió en una recomendación por parte del World Wide Web Consortium (W3C) con el propósito de separar el contenido web de la parte de presentación del mismo.
Las hojas de estilo son de gran utilidad para grandes proyectos de la web, por ejemplo en la edición de un sitio web compuesto de cientos de páginas y referencias, o por ejemplo, la edición de un periódico online que necesita actualizaciones diarias. Así se pueden crear muchas páginas con un diseño homogéneo siendo más fácil su mantenimiento y actualización.

Algunas de las ventajas que este lenguaje trajo consigo son:

- Tamaños de fuentes ilimitados.
- Diferentes bordes para las tablas.
- Posicionamiento de imágenes de forma más precisa.
- Sangrías.
- Etc.

Antes de abordar las Hojas de Estilo, es necesario, si no dominar el lenguaje HTML, al menos comprenderlo y saber manejarlo aún sin saberse de memoria todas las etiquetas.
Por otra parte, este lenguaje ofrece la posibilidad de dotar de accesibilidad a la página web, lo cual quiere decir que, además de poder visualizar los documentos en múltiples dispositivos, también facilita el acceso a personas que sufren problemas de visión, ya que con un solo clic, el tamaño de los caracteres puede hacerse más legible.

2.2. ¿Cómo se aplican las Hojas de Estilo?.

Las Hojas de Estilo permiten redefinir las reglas para presentar una página web. Por ejemplo, si con HTML se puede definir que un titular escrito entre las etiquetas

•••

aparezca mayor que el resto, en negrita, con una fuente Times Roman y alineado a la izquierda, con las Hojas de estilo se puede cambiar la regla que HTML define como predeterminada para este titular. Por ejemplo puede expresarse el texto en azul y con 20 puntos. Existen tres maneras de añadir estilos a las páginas web:

1. Añadiendo instrucciones de estilo sólo a etiquetas concretas, o a un grupo de ellas. La sintaxis general se presenta así:

   ```
   <h1 style="color:blue;"> Un texto cualquiera
   </h1>
   ```

   En este caso las comillas son obligatorias.

2. Incluyendo las instrucciones de estilo en el documento HTML de una página concreta. Esto permite cambiar la apariencia de una hoja entera, cambiando unas pocas líneas. En este caso, el código deberá estar entre las etiquetas.

   ```
   <head> y </head> del documento HTML.
   <head> <style type="text/css">
   h1 { color:blue; }
   </style> </head>
   ```

3. Enlazando todos los documentos HTML de todas las páginas que componen un sitio web con un fichero de definición del estilo. De esta manera, se puede cambiar la apariencia de múltiples páginas retocando un solo fichero. Este fichero contiene únicamente declaraciones de estilo, no puede contener etiquetas de HTML y no puede incluir las etiquetas . El archivo se guarda con la extensión *.css y lo aconsejable es almacerarlo en la misma carpeta que el archivo HTML.
   Una vez creado el documento *.css, para vincularlo al archivo HTML o la página web propiamente dicha, éste deberá incluir entre las etiquetas y un enlace de la siguiente manera:

   ```
   <link rel="stylesheet" href="ejemplo.css">
   ```

Siendo ejemplo.css el fichero creado con los estilos.

Estos estilos se pueden aplicar en el diseño de páginas web independientemente unos de otros o bien mezclando varios métodos entre sí. Incluso, múltiples estilos externos pueden ser referenciados en un solo documento HTML. Cuando hay varios estilos ¿cuál será el que se muestre? A grandes rasgos, podemos decir que todos los estilos caen en una nueva hoja con un "estilo virtual" según las siguientes normas, de las que la número cuatro tiene la mayor prioridad:

1. Un valor por defecto que determine el navegador.
2. Una hoja de estilo externa.
3. Una hoja de estilo interna (en la etiqueta).
4. Un estilo en línea (dentro de un documento HMTL).

Por tanto, un estilo en línea en un documento HTML tiene la prioridad más alta, lo que significa que éste sobreescribirá cualquier otro estilo.

2.2.1. Selectores y declaraciones.

Los selectores son los efectos de estilo que se aplican a un elemento o a una etiqueta de HTML. Su sintaxis se construye colocando el identificador de la etiqueta HTML (sin los símbolos < y >) seguido del estilo encerrado entre los símbolos { y }. Por ejemplo:

```
selector
{estilos}
```

En un ejemplo real, para que un párrafo aparezca con letras en cursiva y en color rojo, la sintaxis sería:

```
p {font: italic; color:red}
```

La declaración sería el contenido entre los símbolos { }. Se construye con el esquema **propiedad: valor;** La propiedad indica el elemento que se definirá con un estilo.
El valor indica la medida en que la propiedad tendrá efecto. Dentro de la declaración pueden introducirse cuantos estilos se deseen separándolos con punto y coma.

```
selector
propiedad1:valor;
propiedad2:valor;
propiedad3:valor; }
```

Un ejemplo real:

```
body {
font : 75% georgia, sans-serif;
line-height : 1.88889;
color : #555753;
background : #fff url(/001/blossoms.jpg) no-repeat
bottom right;
margin : 0;
padding : 0; }
```

Como se puede observar, CSS permite introducir espacios en el código para hacerlo más legible. Además, también admite comentarios del programador para que éste pueda organizar su trabajo. Éstos se introducen en el código entre los símbolos /* y */.
Por ejemplo: /* Comentario */. El contenido del comentario no aparecerá en la página.

2.2.2. Las clases.

Una clase no es más que un tipo de estilo que se aplicará a un grupo de etiquetas.
Para definirlos se utiliza la siguiente nomenclatura:

```
.nombre_clase
{estilos}
```

De esta forma, el estilo no está asignado a ninguna etiqueta en concreto. Para poder hacer referencia a esa clase, y que una etiqueta obtenga dicho estilo, se utiliza el parámetro “class” de la siguiente forma:

```
<etiqueta class="nombre_clase">
```

En un caso real, por ejemplo en un párrafo defi nido en HTML, sería:

```
<p class=”nombre_clase”>
```

De igual forma, se pueden crear clases para etiquetas en concreto, para establecer una serie de estilos a un grupo de etiquetas pero no a todas las etiquetas del mismo tipo. Un ejemplo podría ser el siguiente:

```
<style type="text/css">
A.clase1
{parámetros de estilo para los hiperenlaces de clase 1}
</style>
...
<a href="..." class="clase1">...
```

En este ejemplo se observa que se le asigna un nombre al grupo, en este caso “clase1”, y posteriormente se indica que la etiqueta pertenece a ese grupo mediante el parámetro “class”. De esta forma se podrán crear estilos para grupos de etiquetas del mismo tipo sin obligar a que todas las etiquetas de ese tipo tengan el estilo creado. Por ejemplo, para el caso concreto de la etiqueta ..., el lenguaje HTML define una serie de estilos aplicable a cuatro situaciones determinadas:

- link: Indica el estilo de un enlace que no ha sido visitado.
- visited: Indica el estilo de un enlace que ha sido visitado.
- active: Indica el estilo de un enlace que está siendo pulsado.
- hover: Indica el estilo de un enlace sobre el que está pasando el ratón.

Son las llamadas pseudoclases. También se puede definir el estilo de estas pseudoclases de la siguiente forma:

```
A: nombre_pseudoclase {
...
}
```

E incluso definir grupos de la pseudoclase:

```
A.clase1 : nombre_pseudoclase {
...
}
```

2.2.3. Parámetros para definir los estilos.

Para definir los estilos, las etiquetas se dividen en tres tipos:

1. De bloque: Los que hacen que se empiece en una línea nueva, como

   , las cabeceras, etc.

2. Incrustados en línea: No alteran la línea en la que se encuentran, como ***, , etc.***
3. ***De lista: Son los elementos de una lista delimitados por***
4. ***.***

Todo elemento de bloque se considera rodeado por una caja con propiedades de márgenes, borde, padding y fondo. Además, la caja que lo rodea tiene un cierto ancho y una cierta alineación con respecto al documento. Para escribir una propiedad se utiliza la siguiente notación:

```
Etiqueta.clase {
Propiedad1 : valor1;
Propiedad2 : valor2;
}
```

Todas las longitudes deberán ser de la forma valor unidad, donde valor será un número y la unidad será px (píxeles), cm (centímetros), mm (milímetros), pt (puntos) e in (pulgadas).

Algunas de las propiedades de las etiquetas de tipo bloque son:

- ***margin-left, margin-right, margin-top, margin-bottom, margin: definen en longitud o porcentaje el margen izquierdo, derecho, superior, inferior o respecto a todos los lados, del bloque con relación al resto de los elementos del documento.***
- ***padding-left, padding-right, padding-top, paddingbottom, padding: Definen en longitud o porcentaje el espacio, a la izquierda, derecha, arriba, abajo o respecto a todos los lados, entre el borde y el contenido del bloque.***
- ***border-left-width, border-right-width, border-top-width, borderbottom- width, border-width: Definen en longitud el ancho del borde izquierdo, derecho, superior, inferior o todos.***
- ***border-style: Define el tipo de borde del bloque: none (ninguno), solid (línea), double (doble línea).***
- ***border-color: Define el color del borde del bloque.***
- ***font-size: Define el tamaño de la letra. Normalmente se define en píxeles: "font-size:12px;".***
- ***font-family: Define el tipo de letra, basta con indicar el tipo o lista de tipos de letra que se desean: "font-family:Verdana, Arial;".***
- ***font-weight: Define el grosor de la letra. Puede tomar los valores normal, bolder (negrita grueso), lighter (fina) o bien un valor numérico entre 100 y 900, siendo 100 la letra más fina y 900 la más gruesa.***
- ***font-style: Define el estilo de la letra pudiendo tomar los valores de normal, bold (negrita estándar) o Italic (cursiva).***
- ***text-align: Define la alineación del bloque sobre el que se aplica. Puede tomar los valores left (izquierda), right (derecha), center (centrado) o justify (justificado).***

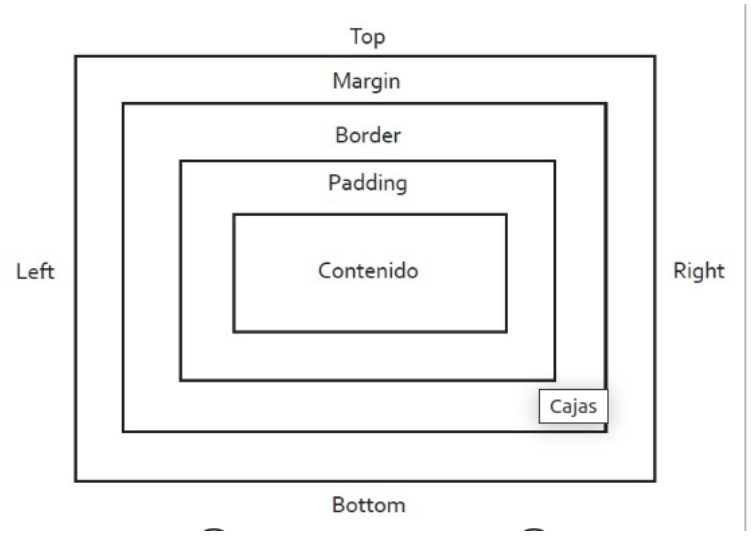
- *text-decoration: Permite definir la decoración de la letra, permitiendo subrayarla o no. Los valores posibles son none, underline (subrayado) o linethrough (tachado).*
- *color: Define el color del texto. Los valores posibles deben ser en código RGB.*
- *background-color: Define el color del fondo del bloque. También debe estar en código RGB.*
- *background-image: Define una imagen que se utilizará de fondo del bloque. Debe tener la URL del fichero de imagen como valor.*

Además de todas estas posibilidades existen algunas más. Si se desea, por ejemplo, crear un estilo con letra en verdana, color rojo, alineada a la derecha y con un borde en color verde se debe especificar:

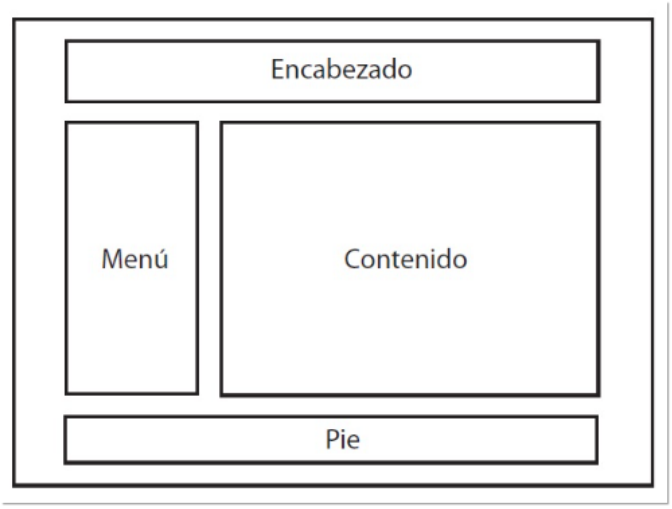
```
.estilo{
    font-family : Verdana;
    font-size : 10px;
    color : #ff 0000; (también se puede poner: “color : red;”)
    text-align : right;
    text-decoration : none;
    border-color : #00ff 00 ; (también se puede poner: “bordercolor: green;”)
    border-width : 2px;
    border-style : solid;
}
```

2.3. La estructura de cajas.

La estructura de cajas es un concepto elemental en CSS y describe las cajas rectangulares que son generadas por los elementos en la estructura de la página web. Cualquier texto o elemento HTML del documento está contenido dentro de una caja. Estas cajas tienen unas propiedades como son la altura y la anchura, los bordes, el relleno (padding) y los márgenes (margin).



De esta manera se sustituye la estructura de tablas que se utilizaba con HTML, la cual estaba bien en un principio, pero conllevaba muchas limitaciones. En el siguiente ejemplo se puede observar cómo la estructura de cajas permite distribuir



En este ejemplo hay que entender que cada elemento (encabezado, menú, contenido y pie) es una caja con sus propiedades de margin, border y padding. Este concepto otorga al diseñador la libertad de controlar totalmente los márgenes, los bordes y cada elemento de la página. Todos ellos se pueden configurar
independientemente.

Para dividir la pantalla en cajas se utiliza la etiqueta

...
. Delante y detrás de estas etiquetas se introduce automáticamente un salto de carro. Además, entre ellas se puede introducir texto, imágenes, etc., y se les puede aplicar estilos independientes. Aunque la etiqueta
...
sirve para crear cajas verticalmente (unas encima de otras), puesto que introduce saltos de carro delante y detrás de donde se sitúe, también se pueden realizar divisiones horizontalmente, tal y como se muestra en el esquema de la imagen anterior. Para ello se utilizan las propiedades clear y float, las cuales son muy útiles para los templates con más de una columna.

float quita al elemento del flujo normal de la página y lo coloca a la izquierda o derecha del resto del contenido de su elemento padre (se lo hace "flotar" hacia uno u otro costado).

clear especifica que un elemento no admite un flotante sobre su izquierda, sobre su derecha o a ambos lados. También se la puede usar en un elemento flotante para impedir que otros flotantes puedan quedar sobre algunos de sus costados.

```
<html>
<head><title>Divs horizontales</title>
<style type="text/css">
#contenedor{
width: 300px;
float: none;
clear: both;
}
#columna{
 background-color:#FFCC99;
 width: 90px;
 float: right;
}
#contenido{
 background-color:#66FFCC;
```

```
 width: 190px;
 float: left;
}
</style>
 </head>
 <div> Encabezado </div>
 <div id="contenedor">
     <div id="columna" class="col">
         test<br />
         test<br />
         test<br />
         test<br />
     </div>
    <div id="contenido" class="cont">
     test<br />
     test<br />
     test<br />
     test<br />
     test<br />
     test<br />
     </div>
 </div>
 <div>
 Pie
 </div>
 </html>
```

En el siguiente link se puede encontrar toda la información publicada por W3C con la relación de propiedades y clases, y la utilización de las mismas para que la página sea accesible.

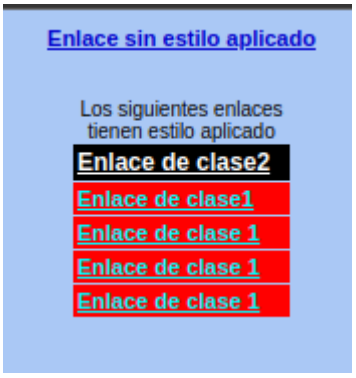
Información publicada por W3C sobre las propiedades y clases: https://www.w3schools.com/css/

2.4. Ejemplo práctico.

A continuación exponemos un ejemplo de la utilización de la tecnología CSS. En este caso se utiliza una técnica bastante popular que define los estilos en los enlaces de una página.
En los enlaces nos encontramos con cuatro situaciones diferentes que hacen referencia a los enlaces no visitados, visitados, los que están siendo pulsados y los enlaces sobre los que se está pasando el puntero del ratón.
Teniendo esto en cuenta, el siguiente ejemplo muestra una página en la que se va a definir el estilo de los enlaces dependiendo de las anteriores situaciones. Para definir el estilo se va a crear una lista de estilos:

```
<html>
   <head> <title>Ejemplo práctico</title>
   <style type="text/css">
      A:link {color:#0000cc;}
      A:visited {color:#0000cc;}
      A:active {color:#0000cc;}
      A:hover {color:#0000ff;}
      A.clase1:visited {color:#00faff;}
      A.clase1:active {color:#ffff00;}
      A.clase1:link {color:#ffff00}
```

```
      A.clase1:hover {color:#00ff00}
      A.clase2:visited {font-size:12;color:#ffffff;}
      A.clase2:active {font-size:12;color:#ffffff;}
      A.clase2:link {font-size:12;color:#ffffff;}
      A.clase2:hover {font-size:12;color:#ffff33;}
      body {font-family:arial;font-size:11;font-weight:bold}
      td {font-family:arial;font-size:11;font-weight:bold}
   </style>
   </head>
 <body bgcolor="#aac8f5"><center>
   <a href="#">Enlace sin estilo aplicado</a>
   <br> <br> <br>
   <span style="font-weight:normal;font-size:10">
      Los siguientes enlaces <br> tienen estilo aplicado
   </span> <br>
   <table width="110" cellspacing="1" cellpadding="2" border="0">
   <tr>
      <td bgcolor="black"><a href="#" class="clase2"> Enlace de clase2</a></td>
   </tr>
   <tr>
      <td bgcolor="red"><a href="#" class="clase1">Enlace de clase1</a></td>
   </tr>
   <tr>
      <td bgcolor="red"><a href="#" class="clase1">Enlace de clase 1</a></td>
   </tr>
   <tr>
      <td bgcolor="red"><a href="#" class="clase1">Enlace de clase 1</a></td>
   </tr>
   <tr>
      <td bgcolor="red"><a href="#" class="clase1">Enlace de clase 1</a></td>
   </tr>
   </table>
 </body></html>
```



Test de altenativa simple/múltiple

1. ***¿Cuál de las siguientes propiedades define el margen izquierdo del bloque en relación con el resto***
   de elementos?
   Margin-left
2. ***¿Cuál de las siguientes propiedades define la longitud del ancho del borde superior en un bloque?***
   border-top-width

3. *¿Cuál de las siguientes propiedades define el tamaño del texto?*
   font-size
4. *¿Cuál de las siguientes propiedades define el grosor de la letra?*
   font-weight
5. *¿Cuál de los siguientes tipos de etiquetas hacen que el estilo comience en una nueva línea?*
   De bloque

2.5. Templates.

Los templates son plantillas en CSS de páginas web completas con un estilo.
Se trata de archivos CSS que pueden ser vinculados al archivo HTML tal y como se explicó en el apartado 2.2. Entre las etiquetas y habría que colocar la siguiente sentencia:

```
<link
rel="stylesheet"
href="template.css">
```

En este ejemplo "template.css" es el nombre del archivo de la plantilla que contiene los estilos.

En la web existen múltiples páginas que ofrecen templates gratuitos descargables, de manera que nosotros sólo tendríamos que realizar el archivo HTML con el contenido de la página y vincular éste al archivo CSS que nos descarguemos.
Un ejemplo de página web para descargar templates sería: https://fct.co/rebranded/

Ideas clave

1. *Mediante la utilización de Hojas de Estilo o CSS se puede separar la programación de un documento HTML de su parte estética.*
2. *Las hojas de estilo se crean asignando una serie de propiedades a una etiqueta concreta, o bien a una clase en concreto. Una clase no es más que un tipo de estilo (tipo de letra, cuerpo, color, etc.) que se aplicará a un grupo de etiquetas.*
3. *Para definirlas se utiliza la nomenclatura:*

   ```
   .nombre_clase
   { estilos
   }
   ```

4. *Para definir los estilos, las etiquetas se dividen en tres tipos:*
   - *De bloque: los que hacen que se empiece en una línea nueva, como*

     , las cabeceras, etc.
   - *Incrustados en línea: no alteran la línea en la que se encuentran, como , , etc.*
   - *De lista: son los elementos de una lista delimitados por*
   - *.*
5. *Para definir estilos se usan entre otros parámetros:*
   - *font-size: Define el tamaño de la letra.*

- font-family: Define el tipo de letra.
- font-weight: Define el grosor de la letra.
- font-style: Define el estilo de la letra.
- text-align: Define la alineación del bloque sobre el que se aplica.
- text-decoration: Permite definir la decoración de la letra, permitiendo subrayarla.
- color: Define el color del texto. Los valores posibles deben ser en código RGB.
- background-color: Define el color del fondo del bloque. También en RGB.
- background-image: Define una imagen que se utilizará de fondo del bloque.

6. Existen tres formas de definir un estilo:
   - De forma directa.
   - Mediante una lista de estilos en el documento HTML.
   - Separar HTML de CSS, referenciando en el documento HTML un fichero que contendrá la lista de estilos.
7. Se puede separar HTML de CSS mediante la referencia de un fichero donde estarán las definiciones de los estilos. Consiste en colocar una referencia en el documento HTML entre las etiquetas: ..., mediante:

```
<link rel="stylesheet" TYPE="text/css" href="hoja.css">
```

3. Javascript

3.1. ¿Qué es JavaScript?.

Javascript es un lenguaje utilizado para dotar a las páginas web de interactividad con el usuario, se introduce en el documento HTML y es ejecutado por el navegador (cliente) del usuario.
La funcionalidad de JavaScript (como la de todos los lenguajes de script) está asociada a "eventos", es decir, sucesos que ocurren en la página web como, por ejemplo, pulsar sobre un icono, pasar el cursor sobre una imagen, enviar un formulario, etc.
Para escribir el código en lenguaje JavaScript se utiliza una etiqueta en HTML que le indica al navegador que el código siguiente se trata de un código script y no de lenguaje HTML.
La etiqueta es .
La etiqueta, además, tiene un atributo para indicar qué lenguaje script se utilizará:

```
<script language="javascript">
[Código JavaScript]
</script>
```

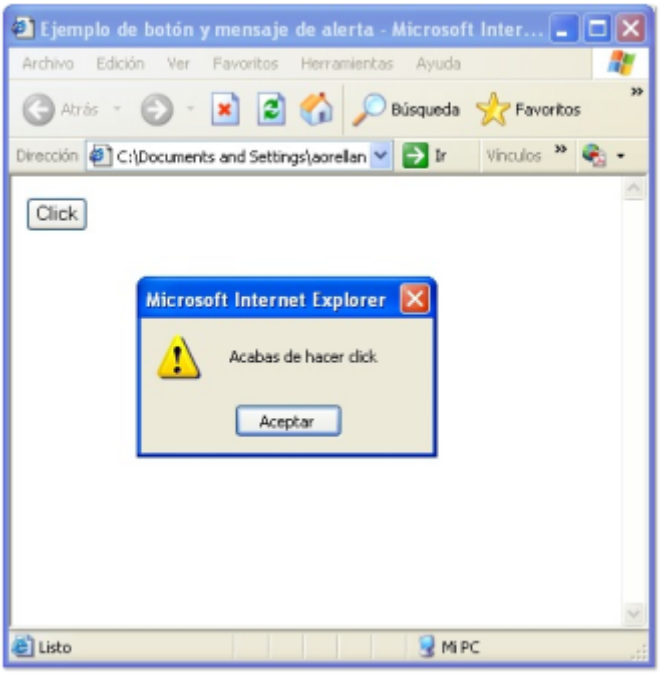
Puede ocurrir que el usuario que ve la página web tenga un navegador antiguo que no entienda el lenguaje JavaScript. Si es así, al visualizar la página, el código JavaScript no se ejecutaría y, en lugar de ello, se vería en la página web como si se tratase de un texto en HTML. Para evitar esto se utilizan las etiquetas HTML de comentario: .
Para escribir un código JavaScript en una página HTML también puede seguirse este esquema:

```
<script type="text/javascript">
<!--
[Código JavaScript]
//-->
```

Las dos barras ("//") colocadas delante del cierre de comentario sirven para el caso contrario, cuando un navegador web entiende el lenguaje JavaScript, si se encuentra con los caracteres "-->" dentro del código JavaScript, daría un error.
Para que este problema no surja, se utiliza la etiqueta de comentario en lenguaje JavaScript que es precisamente: "//".
Un código script puede colocarse también en cualquier lugar del documento HTML.
Sólo hay que tener en cuenta que las sentencias de código JavaScript se van a ir ejecutando a medida que vayan apareciendo.

Por ejemplo, la siguiente función de lenguaje JavaScript sirve para hacer que al usuario le aparezca en pantalla un botón que al ser pulsado muestra un mensaje de información: alert ("texto").

```
<html>
<head>
<input type="submit" onclick="alert 'Acabas de hacer click');return false;"
value="Click">
</body>
</html>
```



Las sentencias se ejecutan linealmente: primero se crea el párrafo, después la línea de JavaScript con la función alert y finalmente se crea el segundo y último párrafo.
Ahora bien, en JavaScript se pueden crear funciones propias de tal forma que el código contenido en ellas no se ejecuta en el lugar del documento HTML en el que está situado, sino que se ejecutan cuando aparezca una llamada a la función.
Sin entrar en detalles, que se verán más adelante, un documento HTML con funciones JavaScript sería:

```
<html><head>
<script type="text/javascript">
    function message()
    {
        alert("Esta caja de alerta se ejecuta con el elemento onload");
    }
</script>
</head>
<body onload="message()">
</body>
</html>
```

En este ejemplo se ha definido la función "aviso" que consiste en presentar al usuario un mensaje de aviso mediante la función alert. Cuando se carga la página aparece el mensaje de alerta.
Al programar en JavaScript, hay que tener en cuenta que es un lenguaje "Case Sensitive", es decir, sensible a las mayúsculas. Esto quiere decir que en el lenguaje JavaScript se interpreta "Aviso" y "aviso" como elementos diferentes.
Resumiendo, las principales características de este lenguaje son:

- *JavaScript fue diseñado para dotar de interactividad a las páginas HTML.*
- *JavaScript es un lenguaje de scripts.*
- *Un lenguaje de scripts es un lenguaje de programación para aplicaciones ligeras.*
- *JavaScript suele ir embebido en el código HTML.*
- *JavaScript se ejecuta sin tener que ser compilado.*
- *No es necesario adquirir una licencia para usar JavaScript.*

3.2. El código JavaScript.

3.2.1. Sentencia y declaraciones.

JavaScript no es lo mismo que Java.
El código JavaScripts que se encuentre entre las etiquetas se ejecutará cuando se cargue la página.
El código JavaScript que se encuentre entre las etiquetas y se ejecutará cuando se le "llame a la acción".
Toda línea de comandos, excepto funciones y bucles, suelen acabar con el carácter ";", aunque esto es opcional.
Todas las funciones y bucles empiezan y acaban con los caracteres "{" y "}".

```
{
    alert("Esta caja de alerta se ejecuta con el elemento onload");
}
```

El lenguaje JavaScript es un lenguaje de programación y, como tal, es conveniente tener algunas nociones de programación antes de introducirse plenamente en su uso.
En este sentido, a continuación se analizan diversas funciones y reglas básicas para permitir un primer acercamiento a este lenguaje.
Una sentencia en JavaScript es un comando para el navegador y su función es decirle lo que tiene que hacer. Por ejemplo, la siguiente sentencia le dice al navegador que escriba "Hola a todos" en la página web:

```
document.write("Hola a todos");
```

Cada sentencia será ejecutada por el navegador en el orden en el que hayan sido escritas:

```
<html><title>Ejemplo JavaScript</title>
<head></head>
<body>
<script type="text/javascript">
    document.write("<h1>Esto es un encabezado</h1>");
    document.write("<p>Esto es un párrafo.</p>");
    document.write("<p>Esto es otro párrafo.</p>");
</script>
</body>
</html>
```

Las sentencias se pueden agrupar en bloques utilizando los símbolos { y }. Los bloques de sentencias sirven para hacer que la secuencia de sentencias se ejecuten juntas. Entre las sentencias más importantes de un lenguaje de programación y, por tanto, de JavaScript, están las sentencias iterativas y la sentencia "if":

```
if (condición) { sentencias1 } else {
sentencias2}
```

Es la sentencia "si condición, entonces sentencias1. En caso contrario, sentencias1", se evalúa la condición. Si ésta es "true" se ejecutan las sentencias marcadas como "sentencias1", y si la condición es "false" se ejecutan las sentencias marcadas como "sentencias2".
Nótese que la condición siempre va entre paréntesis y que tanto las sentencias1 como las sentencias2 empiezan y acaban con una "{" y "}".

```
while (condición) { sentencias }
```

Se trata de una sentencia iterativa donde las "sentencias" se ejecutan hasta que la "condición" deja de cumplirse. Es muy importante que esta "condición" deje de cumplirse en algún momento por acción de alguna de las "sentencias" para que este bucle no sea infinito. Del mismo modo que en la sentencia "if", la condición va entre paréntesis y las sentencias entre "{" y "}".

```
for (expresion_inicial ; condicion; incremento) {
sentencias }
```

Se trata de otra sentencia iterativa. El proceso de ejecución es el siguiente: se ejecuta la "expresion_inicial", se evalúa la "condición" y si es "true" se sigue adelante, se ejecutan las "sentencias", se ejecuta "incremento" y se vuelve a evaluar la condición repitiéndose el ciclo. Aquí es importante que en algún momento la condición sea "false" para dar por finalizada la iteración.
Se ha comentado anteriormente que el lenguaje JavaScript está "guiado por eventos". Esto quiere decir que la forma natural de llamar a las funciones JavaScript es a través de un evento.

A continuación se enumeran los más importantes:

- *Onclick="sentencia" (significa "al pulsar"): cuando se pulsa sobre el elemento HTML cuya etiqueta contiene este atributo, se ejecuta la "sentencia".*
- *Onmouseover="sentencia": al pasar el cursor sobre el elemento HTML cuya etiqueta tiene este atributo, se ejecuta la "sentencia".*
- *Onsubmit="sentencia": al enviar un formulario cuya etiqueta tiene este atributo se ejecuta la "sentencia".*
- *Onload="sentencia": al cargarse la página se ejecuta la "sentencia".*
- *Onunload="sentencia": al salir de la página mediante cualquier hiperenlace, se ejecuta la "sentencia".*

Estos eventos son atributos de las etiquetas HTML y, como tales, se deben especificar así:

```
<script type="text/javascript">
    document.write("<h1>Esto es un encabezado</h1>");
    document.write("<p>Esto es un párrafo.</p>");
    document.write("<p>Esto es otro párrafo.</p>");
</script>
<img src=botoncito.jpg onmouseover="alert('Un evento Onmouseover');">
```

Esto es un encabezado

Esto es un párrafo.

Esto es otro párrafo.

localhost dice

Un evento Onmouseover

Aceptar

3.2.2. Variables.

Una variable en JavaScript se utiliza para trabajar con valores o expresiones. Deben comenzar por una letra o un carácter subrayado. Los valores de la variable pueden cambiar durante la ejecución de un script, pudiéndonos referir a ellas a través de su nombre o cambiando su valor. Como en el caso de cualquier lenguaje de programación, el lenguaje JavaScript utiliza una serie de sentencias y bucles que podrán utilizarse para realizar cualquier funcionalidad. También utiliza variables que deben ser definidas en el momento de asignarles un valor, aunque al tratarse de un lenguaje débilmente tipado no hay que asignarles un tipo determinado. Esto significa que no se vincula una variable a un tipo de datos determinado, sino que una variable puede ser utilizada en primera instancia como un número y posteriormente utilizarse como una cadena de texto, no teniendo que definir desde el principio si se trata de un número y de una cadena de texto.

Para definir una variable se utiliza el comando "var":

```
var edad="23";
```

Donde se define la variable "edad" con un valor de "23" y podrá ser utilizada en todo el código JavaScript del documento HTML. Veámoslo mejor en el siguiente ejemplo:

```
<html>
   <meta charset="utf-8">
   <title>Ejemplo JS</title>
<body>
   <script type="text/javascript">
      var apellido
      apellido="Sánchez";
      document.write(apellido);
      document.write("<br />");
      apellido="Pérez";
      document.write(apellido);
   </script>
   <p>Este script declara una variable, le asigna un valor, la muestra,
   el valor y la muestra otra vez.</p>
</body>
</html>
```

3.2.3. Objetos.

Al tratarse de un lenguaje orientado a objetos, el lenguaje utilizará dichos objetos que no son más que estructuras "contenedoras" de valores, procedimientos y funciones. Para crear un objeto en JavaScript se utiliza la sentencia:

```
Obj = new Object();
```

donde "Obj" contendrá un objeto "Object".
Y para utilizar una función o método de ese objeto se utiliza la siguiente nomenclatura:

```
Obj.funcion
```

Existe una serie de objetos que no necesitan ser creados:

- *Objetos del lenguaje, relacionados con la estructura del lenguaje.*
- *Objetos del navegador, relacionados con el navegador y las posibilidades de programación HTML.*

El objeto más importante del lenguaje es el objeto string. Se trata de un objeto para el manejo de cadenas de texto. Cuando se asigna una cadena de texto a una variable, el lenguaje JavaScript crea un objeto "string" para esa variable. Así se puede obtener la longitud de la cadena mediante la propiedad "length":

```
var cadena = "Esta es mi cadena"
var longitud = cadena.length
```

transformar la cadena a mayúsculas:

```
cadena.toUpperCase()
```

o a minúsculas:

```
cadena.toLowerCase()
```

Otros objetos del lenguaje son: array, math, date, boolean o number, en los cuales no se profundizará por tratarse de un objeto de estudio que se desvía de la funcionalidad de este manual.
Los objetos de navegador son objetos que manejan el contenido de los documentos HTML y las ventanas del navegador. Por tanto, estos objetos siguen una jerarquía como la siguiente:

- *window*
  - *history*
  - *location*
  - *document*
    - *anchor < a name="..."> ...*
    - *area ...*
    - *form*

- *button*
- *checkbox*
- *hidden*
- *password*
- *radio*
- *reset* Restablecer
- *select*
- *submit* Enviar
- *text*
- *textarea* ...

- *image*
- *link < a href="..."> ...*
- *frame*

En esta jerarquía puede observarse cómo dependen unos objetos de otros y qué objeto se refiere a cada etiqueta HTML.

Por ejemplo, si se desea hacer referencia a un campo de texto de un formulario concreto se utilizará la
siguiente sentencia JavaScript:

```
ventana.documento.formulario.campo
```

"ventana" es el nombre de la ventana (por defecto "window"), "documento" el nombre del documento
(por defecto "document"), "formulario" el nombre del objeto forms y campo es el nombre de la etiqueta
input.
Normalmente, mientras no se trabaje con varias ventanas o frames, la referencia al objeto ventana se
puede omitir.
Cada uno de estos objetos tiene una serie de propiedades y una serie de métodos o funciones. En este
manual sólo se hará referencia a las más relevantes y útiles del objeto window que permiten abrir o
cerrar ventanas del navegador:

- *window.open(URL): abre una ventana nueva y carga en ella la dirección indicada en la URL.*
- *window.close(): cierra la ventana en la que se encuentra el usuario.*

Las funciones pueden necesitar parámetros. Estos vendrán definidos dentro de paréntesis como se
indica en el siguiente ejemplo:

```
function funcion(parametro1, parametro2,...)
{
...
}
```

Y al llamar a la función, se indicarán los valores en el mismo orden:

```
funcion(valor1, valor2,...);
```

Aunque una función no lleve parámetros, de igual forma debe llevar los paréntesis (aunque éstos aparecerán vacíos):

```
funcion();
```

Una vez expuestos estos objetos, es el momento de analizar los diversos operadores del lenguaje script.

3.2.4. Operadores.

Los operadores son símbolos o combinaciones de símbolos que actúan sobre varios elementos realizando una función y devolviendo un valor.
Ejemplo
El símbolo "+" es un operador que actúa sobre los elementos que tiene delante y detrás (5 y 3, por ejemplo), realiza la función de sumar ambos elementos y devuelve el resultado de la suma: 5+3=8.
Los operadores se clasifican en:

- **Operadores aritméticos:** siendo y=7

Operador	Descripción	Ejemplo	Resultado
+	Suma	x=y+2	x=9
-	Resta	x=y-2	x=5
*	Multiplicación	x=y*2	x=14
/	División	x=y/2	x=3.5
%	Resto de la división	x=y%2	x=1
++	Incremento	x=++y	x=8
--	Decremento	x=--y	x=6

- **Operadores de comparación:** siendo x=7

Operador	Descripción	Ejemplo
==	Igual a	x==8 es falso
===	Exactamente igual a	x=="7" es falso x==7 es verdadero
!=	No es igual a	x!=8 es verdadero
>	Es mayor que	x>5 es verdadero
<	Es menor que	x>5 es falso
>=	Es mayor o igual que	x>=7 es verdadero
<=	Es menor o igual que	x<=8 es falso

- **Operadores lógicos:** siendo x=1 e y=2

Operador	Descripción	Ejemplo
&&	y	(x < 10 && y > 1) es verdadero
||	o	(x==5 || y==5) es falso
!	no	!(x==y) es verdadero

- **Operadores de asignación:** asigna valores en función del operador. Para x=10 e y=5.

Operador	Descripción	Es lo mismo que	Ejemplo
=	x=y		x=5
+=	x+=y	x=x+y	x=15
-=	x-=y	x=x-y	x=5
*=	x*=y	x=x*y	x=50
/=	x/=y	x=x/y	x=2
%=	x%=y	x=x%y	x=0

Ejemplo:

```
<script type="text/javascript">
x=10+5;
document.write(x); document.write("<br />");
x="8"+"6";
document.write(x); document.write("<br />");
x=5+"2";
document.write(x); document.write("<br />");
x="5"+3;
document.write(x); document.write("<br />");
 </script>
 <p>Si se suma un número a una cadena ("5"),
 el resultado será una cadena.</p>
```

3.2.5. Arrays.

Los Array son unos tipos de variables donde podemos introducir múltiples valores, al contrario que en las variables comunes, dónde sólo podemos introducir un valor. Podemos acceder a dichas variables de manera independiente, y para ello se utilizan los índices, en los que se especifica la posición de dichas variables.
Para crear los Arrays debemos seguir los siguientes pasos:

- ***Crear el array:***

  ```
  var miArray = new Array()
  ```

  Dentro de los paréntesis podemos especificar un número que indicará el número de posiciones del
  Array. La palabra Array siempre ha de llevar la primera letra en mayúsculas.

- ***Introducir datos:***

  Indiquemos o no el número de casillas del array, podemos introducir en el array cualquier dato. Si la
  casilla está creada se introduce simplemente y si la casilla no estaba creada se crea y luego se introduce el dato, con lo que el resultado final es el mismo. Esta creación de casillas es

dinámica y se produce al mismo tiempo que los scripts se ejecutan.
Ejemplo:

```
miArray[0] = 290
miArray[1] = 97
miArray[2] = 127
```

Los arrays empiezan siempre en la posición 0, así que un array que tenga por ejemplo 10 posiciones, tendrá casillas de la 0 a la 9.
Para recoger datos de un array lo hacemos igual: poniendo entre corchetes el índice de la posición a la que queremos acceder.
Podemos almacenar cualquier tipo de datos en las casillas de los Array, ya sean caracteres, números, textos, boleanos, etc.

```
miArray[0] = "Hola"
miArray[1] = "123"
miArray[2] = "true"
```

3.3. Ejemplo práctico: acceso a una página mediante contraseña.

En este ejemplo desarrollaremos un mensaje de secuencia de comandos que solicite una contraseña para acceder a una página web. Para ello recurriremos a una caja de diálogo del tipo "prompt", la cual muestra un diálogo de campo de formulario con botones Aceptar y Cancelar, un texto definido por el primer parámetro enviado a la función y un input de texto con valor predeterminado definido por el segundo parámetro. En este caso el, el primer texto definido será "Introduce la contraseña" y el segundo, el que pertenece al campo de texto donde debe introducirse la contraseña, estará vacío:

```
password=prompt("Introduce la contraseña","");
```

"Prompt" nos será útil sobre todo para recojer datos del usuario para utilizar en nuestro script en tiempo de ejecución. El parámetro "while" es un parámetro de condición, en el especificaremos cuál será la contraseña que debe insertar el usuario, que en este caso será la cadena de texto "contraseña". Si el usuario inserta bien dicha contraseña, pulsa aceptar sin rellenar la contraseña, o pulsa en cancelar, el bucle devolverá una ventana de alerta:

```
alert("Introduzca la contraseña correcta");
```

Esto es así porque "while" es la condición para saltar el bucle. Si la contraseña no es la indicada en el parámetro "while", aparecerá el otro "Prompt".

Veamos el código completo:

```
<html>
<head>
   <meta charset="utf-8">
```

```
</head>
<title>Acceso a página con contraseñas</title>

<script lenguage="javascript">
    password=prompt("Introduce la contraseña","");
    while(password!="contraseña"){
        alert("Introduzca la contraseña correcta");
        password=prompt("Debes introducir la contraseña para acceder a esta
página","");
     }
 </script>
 </head>
 <body>
 <h1>Enhorabuena, has conseguido entrar en la página</h1>
 </body>
 </html>
```

Rellena los huecos escribiendo la palabra o expresión correcta para completar cada frase.

1. ***Para escribir el código en javascript usamos la etiqueta SCRIPT que indica al navegador que el código siguiente es un script.***
2. ***El lenguaje JavaSript utiliza sentencias y bucles que podrán utilizarse para realizar cualquier funcionalidad.***
3. ***Un script puede colocarse en cualquier lugar del documento html.***
4. ***Las variables pueden ser utilizadas en primera instancia como un número y posteriormente utilizarse como una cadena de texto.***

Ideas clave

1. ***JavaScript es un lenguaje de tipo script compacto, orientado a objetos y guiado por eventos. Mediante ciertos objetos del lenguaje se puede controlar en Javascript diversas partes del documento HTML, y así controlar el funcionamiento de la página web mediante ciertos eventos provocados por el usuario, como pasar con el cursor por una zona determinada, pulsar en un lugar determinado, y otros.***
2. ***Para introducirlo en un documento HTML, se utiliza la etiqueta .***
3. ***Para trabajar con el código Javascript hay que tener en cuenta dos premisas muy importantes:***
    1. ***Las líneas de comandos, excepto funciones y bucles, acaban con el carácter ";".***
    2. ***Todas las funciones y bucles empiezan y acaban con los caracteres "{" y "}".***
4. ***El lenguaje JavaScript es un lenguaje de programación y, como tal, utiliza una serie de sentencias y bucles para realizar cualquier funcionalidad. También utiliza variables ("var") que deben ser definidas en el momento de asignarles un valor, aunque al tratarse de un lenguaje débilmente tipado no hay que asignarles un tipo determinado.***
5. ***Al tratarse de un lenguaje orientado a objetos, JavaScript utilizará dichos objetos. Existe una serie de objetos que no necesitan ser creados:***

    1. *Objetos del lenguaje, relacionados con la estructura del lenguaje: el más importante es el objeto String, que se utiliza para el manejo de cadenas de texto.*
    2. *Objetos del navegador, relacionados con el navegador y las posibilidades de programación HTML. Cada uno de estos objetos tiene una serie de propiedades y de métodos o funciones.*
6. *El lenguaje Javascript cuenta también con una serie de operadores:*
    1. *Operadores aritméticos.*
    2. *Operadores de comparación.*
    3. *Operadores lógicos.*
    4. *Operadores de asignación.*
7. *Un uso muy habitual del lenguaje Javascript es la comprobación de valores de formularios. También es habitual ver menús u otros elementos interactivos programados mediante este lenguaje.*

4. HTML dinámico o DHTML.

4.1. Introducción a DHTML.

En principio, destacar que DHTML no es un lenguaje de programación nuevo sino la combinación de HTML, CSS, DOM (Document Object Model) y JavaScript. Su objetivo es aumentar la funcionalidad de un sitio web en aquellos casos en los que HTML no es suficiente, como crear determinados documentos interactivos y multimedia. Así, con DHTML se pueden crear animaciones, menús desplegables, mostrar y ocultar partes de una página, etc. Incluso permite, una vez se haya cargado la página en el navegador, modificar, cambiar la apariencia ocultar, mostrar y animar el contenido dinámicamente. DHTML tiene la ventaja de que no requiere de plug-in para utilizarlo.
DHTML se compone de:

- *CSS: permite especificar atributos estilísticos de los elementos tipográficos de las páginas.*
- *Posicionamiento de contenidos: cada componente de la página web se mostrará en el lugar que se desee de ella.*
- *Fuentes cargables: permite asegurar que la página se mostrará con las fuentes con las que se diseñó. Para ello, habrá que especificar e incluir el tipo de fuente en el documento HTML. Los lenguajes de script como Javascript o VBScript tienen acceso a todas las propiedades CSS a través del modelo de objeto documento (DOM), es decir, un estilo aplicado a un documento puede cambiar cuando el ratón pase sobre él, o un elemento posicionado estáticamente puede moverse por la página.*
  Por tanto, para trabajar con DHML es necesario conocer y entender el concepto de DOM.

Definición:

Este documento es simplemente un "armazón" que contiene los objetos que aparecen en el navegador, tales como imágenes, formularios, capas, etc. Los desarrolladores emplean este "armazón" o estructura jerárquica para manipular y acceder a lo que aparece en pantalla en la página.

Un ejemplo simple sería cómo acceder a un input box de un formulario:

```
document.formName.inputName
```

Donde formName sería el nombre de nuestro formulario y inputName sería el nombre de nuestro input box.
Otro ejemplo:

```
document.formName.elements[i]
```

Aquí formName sería el nombre del formulario y i sería el número del elemento en ese formulario, al cual queremos acceder.

En un ejemplo real de formulario, acceder a los elementos sería de la siguiente manera:

```
<form name="myForm">
<input type="text" name="textBox1">
</form>
```

Ahora accedamos a este input de varias formas colocando una alerta que aparezca en cada cambio de opción para ver qué ocurre:

```
<form name="form1">
<select name="select" onchange="alert(eval(this[this.selectedIndex].text))">
<option selected>document.forms[0].textBox1</option>
<option>document.forms[0].elements[0]</option>
<option>document.myForm.textBox1</option>
<option>document.myForm.elements[0]</option>
</select>
</form>
```

En primer lugar, estamos accediendo al objeto ventana, en segundo lugar al objeto documento, en tercer lugar al objeto "myForm" y en último lugar estamos accediendo al elemento textBox1 del formulario.
Pulsando en cada opción del desplegable aparecerá la siguiente alerta:

Una vez que hemos accedido al objeto textBox1, podemos acceder a sus propiedades.
Con una imagen sería de la siguiente manera:

```
document.images[i]
```

O bien:

```
document.images[imageName]
```

O bien:

```
document.imageName
```

De acuerdo con W3C, el DOM es una interfaz de programación de aplicaciones (API) para documentos
validos HTML y XML. Define la estructura lógica de los documentos y el modo en que se accede y se
manipula su contenido.
El DOM permite un acceso a la estructura de una página HTML mediante el mapeo de los elementos en un árbol de nodos. Cada elemento se convierte en un nodo y cada porción de texto en un nodo de texto. Gracias a Javascript podemos acceder a cada uno de los elementos de una página utilizando tan sólo algunos métodos y propiedades. Si deseamos encontrar de manera rápida y fácil un elemento podemos utilizar el método getElementById, que permite un acceso inmediato a cualquier elemento
tan solo conociendo el valor de su atributo id:

```
<p>
    <a id="contacto" href=”contactos.html”>Contáctenos</a>
</p>
var elementoContacto = document.getElementById("contacto");
```

Los atributos de las etiquetas HTML están disponibles como propiedades de sus correspondientes nodos en el DOM, y estas propiedades de estilo pueden ser aplicadas mediante el DOM. Cada atributo CSS posee una propiedad del DOM equivalente, formándose con el mismo nombre del atributo CSS pero sin los guiones y llevando la primera letra de las palabras a mayúsculas. Véase el siguiente ejemplo para mayor entendimiento donde se utiliza un atributo CSS modelo:

```
algun-atri-buto-css
```

Tendrá como equivalente la siguiente propiedad o método en Javascript:

```
algunAtributoCss
```

Por tanto, para cambiar el atributo CSS font-family de un elemento, podría realizarse de lo siguiente:

```
ancla.style.fontFamily = ‘sans-serif’;
```

En muchos casos es necesario hacer aparecer y desaparecer un determinado elemento, para lo cual
se utiliza el atributo CSS display, por ejemplo, para hacerlo desaparecer:

```
ancla.style.display = ‘none’;
```

En resumen, explorando el DOM se puede encontrar, cambiar, adicionar y eliminar elementos de un
documento. Es sin duda una una poderosa técnica para generar aplicaciones web dinámicas en el lado
del cliente. De hecho, se ha vuelto imprescindible para trabajar con la web 2.0.

Por otra parte, para no estar tecleando el código JavaScript en la manipulación del DOM, se puede recurrir a herramientas como Prototype, una librería JavaScript que permite el DOM scripting.

En el siguiente ejemplo podremos ver cómo animar un texto tocando solamente el DOM:

```
<html>
   <head>
     <title>Ejemplo de animación de texto con DOM</title>
   <script>
      defaultString="HOLA";
      fontsize=120;
      posleft=120;
      postop=50;
      vala=160;
      valc=255;
      function q1() {
         for(i=0;i<11;i++) {
            node=document.getElementById("area");
            beforediv=document.createElement("DIV");
            cc1=255*(i/11);
            cc2=160*(i/11);
            gfx1=parseInt(vala-cc2);
            gfx2=parseInt(valc-cc1);

            if(i!=10) {
               str="position:absolute;top:"+postop+"px;left:"
               +posleft+"px;color:rgb("+gfx1+","+gfx1+","
               +gfx2+");width:860px;height:600px;font-size:"+fontsize+"pt;";
            }
            else {
               str="position:absolute;top:"+postop+"px;left:"
               -+posleft+"px;color:rgb(0,0,0);width:860px;height:600px;font-
size:"
               +fontsize+"pt;";
            }
            if (navigator.userAgent.indexOf("Gecko")>-1)
               beforediv.setAttribute("style",str);
            else
               beforediv.style.cssText = str;
```

```
                beforediv.setAttribute("id","object"+i);
                newText=document.createTextNode(defaultString);
                beforediv.appendChild(newText);
                node.appendChild(beforediv);
                fontsize+=2;
                posleft+=5;
                postop-=(3-((i/20)*2));
            }
            setTimeout("a3danimation()",100);
        }
        ox=100;
        oy=100;
        pi=3.141516*2;
        ccounter=0;
        ww=1;
        function a3danimation() {
            ww+=.1;
            ccounter++;
            if(ccounter<70 ) {
                for(i=0;i<11;i++) {
                    pis=pi*(ccounter/70);
                    ll=(Math.sin(pis))*50*(i/ww);
                    tt=(Math.cos(pis))*50*(i/ww);
                    posx=ox+ll-i*2;
                    posy=oy+tt-i*2;
                    document.getElementById("object"+i).style.left=posx+"px";
                    document.getElementById("object"+i).style.top=posy+"px";
                }
                setTimeout("a3danimation()",30);
                }
            else {
                ccounter=0;
                setTimeout("a3danimation()",30);
            }
        }
    </script>
    </head>
    <body onload="q1()" bgcolor="WHITE" text="white">
        <div id="area" style="position:absolute; left:100px;
top:050px;color:rgb(0,0,255)">
        </div>
    </body>
  </html>
```

4.2. Definición de capa (layer).

Aunque ya explicamos la etiqueta

...
anteriormente, vamos a volver a abordarla para ver su relación con DHTML. En un documento HTML se pueden definir unos elementos denominados capas que tienen la función de definir divisiones (bloques funcionales rectangulares) a las que se le pueden aplicar estilos de forma independiente al resto del documento. Todos los atributos de las capas se pueden modificar, ya que cada capa funciona como parte independiente en una página web. Existen dos etiquetas

para definir las capas:

```
<ilayer>… </ilayer> y <div> … </div>
```

Las dos tienen el mismo funcionamiento y atributos pero la primera es compatible solamente con el navegador Netscape Communicator (está obsoleta), por lo tanto, es preferible utilizar solamente la segunda y, de esta forma, asegurarse la compatibilidad con todos los navegadores. De esta forma se podría, por ejemplo, colocar una imagen en cualquier parte de un documento HTML, de la siguiente forma:

```
<div><img src="foto.gif" width="50">
</div>
```

Como se puede observar, la etiqueta no tiene ningún atributo y, por tanto, esta capa estará colocada dentro del documento HTML justo en el sitio en el que está escrita: si se coloca una tabla y después se escribe este código de capa, la imagen aparecerá debajo de la tabla.
Para poder colocar la capa en un lugar determinado, se utilizarán estilos siguiendo la nomenclatura utilizada para la hoja de estilos, tal y como se verá en el siguiente apartado, incluyendo algún otro parámetro propio de las capas como, por ejemplo, la posición de la capa en la página web, el tamaño de la capa o si se trata de una capa visible o no.

4.3. Propiedades de las capas.

Los atributos que se pueden utilizar para la etiqueta de capas

…
son principalmente dos:

- *Id: Se utiliza para darle una identificación, es decir, un nombre, a la capa. De esta forma la capa puede ser referida desde un programa JavaScript desde el que se podrá modificar sus atributos dinámicamente.*
- *Style: Con el que se definirán todos los estilos que se deseen para la capa en concreto. Estos estilos siguen la nomenclatura estudiada en el tema anterior. Así se puede dar un estilo a la letra de la capa, a las tablas de la capa, etc.*

Por tanto, a la hora de utilizar estos parámetros con las etiquetas de capa se escribiría:

```
<div id="capaprueba" style="font-size:20 px;color:red;
background-color:blue; border-width:10 px">
Esto es una capa de prueba con la letra en color rojo
<br> y 10 píxeles de tamaño
</div>
```

De esta forma se ha definido para la capa "capaprueba" que el tamaño de letra sea de 20 píxeles y de color rojo.

Existen varios parámetros de estilo propios de la capa. Son los siguientes:

- *Position: indica el tipo de posicionamiento de la capa, es decir, si los valores de posición asignados están referidos a la página web o bien se refieren al lugar del documento HTML en el que está escrito el código de la capa. Dos posibles valores:*

    - *Relative, la posición de la capa estará tomada desde el punto de la página web desde la que está escrito el código de la capa. De esta forma, si se define una posición de 100 píxeles a la derecha y 100 píxeles hacia abajo, la capa se situará a esa distancia del punto en el que estaba colocada anteriormente.*
    - *Absolute, la posición se establece con respecto al principio de la página, sin importar dónde está definida la capa en el documento HTML. Así, los 100 píxeles a la derecha y abajo se aplicarán desde el punto de comienzo de la página web.*
- *Top: indica la distancia en vertical en la que se colocará la capa. Si el parámetro position era relative, esta distancia se aplicará desde el punto en el que se definió la capa. Pero si el parámetro position era absolute se refiere a la distancia respecto al punto superior de la página.*
- *Left: indica la distancia horizontal respecto al lado derecho. Si el parámetro position era relative, esta distancia se aplicará desde el punto en el que se definió la capa. Si por el contrario, el parámetro position era absolute se refiere a la distancia respecto al margen derecho de la página web.*
- *Height: define la altura de la capa, es decir, su tamaño en sentido vertical. Debe estar especificado en píxeles y debe ser suficiente para que los elementos introducidos en la capa quepan en dicho espacio.*
- *Width: define la anchura de la capa en píxeles. Al igual que el parámetro Height, la medida debe ser suficiente para albergar los elementos definidos dentro de la capa.*
- *Visibility: define si la capa en cuestión será visible o, por el contrario estará oculta al usuario. Puede tener tres posibles valores:*
    - *visible, de esta forma la capa será visible en la página web.*
    - *hidden, mantiene oculta la capa al usuario.*
    - *inherit, este es el parámetro por defecto e indica que toma la propiedad de visibilidad de la*
      capa en la que está contenida la capa actual. Es decir, si se define una capa que está dentro de otra capa, la capa interior tomará el parámetro visibility de la capa en la que está contenida. Si no existiese una capa que contenga a la actual, se considera que la capa está contenida en la página web que es siempre visible y, por tanto, la capa también lo es.
- *Z- Index, sirve para indicar la posición de la capa respecto al eje z, es decir, definir la "altura"*
  que tendrá la capa en cuanto a la profundidad de la página web. De esta forma se pueden colocar unas capas encima de otras. El parámetro es un número que indica la altura y, por tanto, a mayor número próxima se verá la capa.
- *Clip, tiene como objetivo "recortar" varias partes de la capa y que éstas no sean visibles. De esta forma sólo se visualizará la parte de la capa que se desee, e incluso, haciendo referencia a dicha parte de la capa desde un programa de Javascript, se podrá modificar esta parte visible consiguiendo efectos muy interesantes.*

  Para utilizar este atributo se especificará:

```
<div styles="Clip:rect(top,right,bottom,left);">
```

Los 4 parámetros que necesita esta función definen los 4 valores necesarios para definir el rectángulo
de la capa que será visible. Los valores que se deben indicar son:

- *"top": indica la distancia en píxeles del margen superior del recuadro respecto al margen superior de la capa.*
- *"right": indica la distancia en píxeles del margen derecho del rectángulo respecto del margen*
  izquierdo de la capa.
- *"bottom": indica la distancia en píxeles del margen inferior del rectángulo respecto del margen*
  inferior de la capa.
- *"left": indica la distancia en píxeles del margen izquierdo del rectángulo respecto del margen izquierdo de la capa.*

Gráficamente resulta más fácil observar a qué hacen referencia estos valores:
Estos son los parámetros de estilo propios de las capas. Se pueden establecer los estilos de la capa de
dos formas:

- *Mediante la etiqueta de la capa:*

```
<div style="left:50; top:100;"> ...
</div>
```

- *A través de una hoja de estilos con parámetros generales de las capas:*

```
<style type="text/css">
div.capa1 {
    left:50;
}
</style>

<div class="capa1">...</div>
```

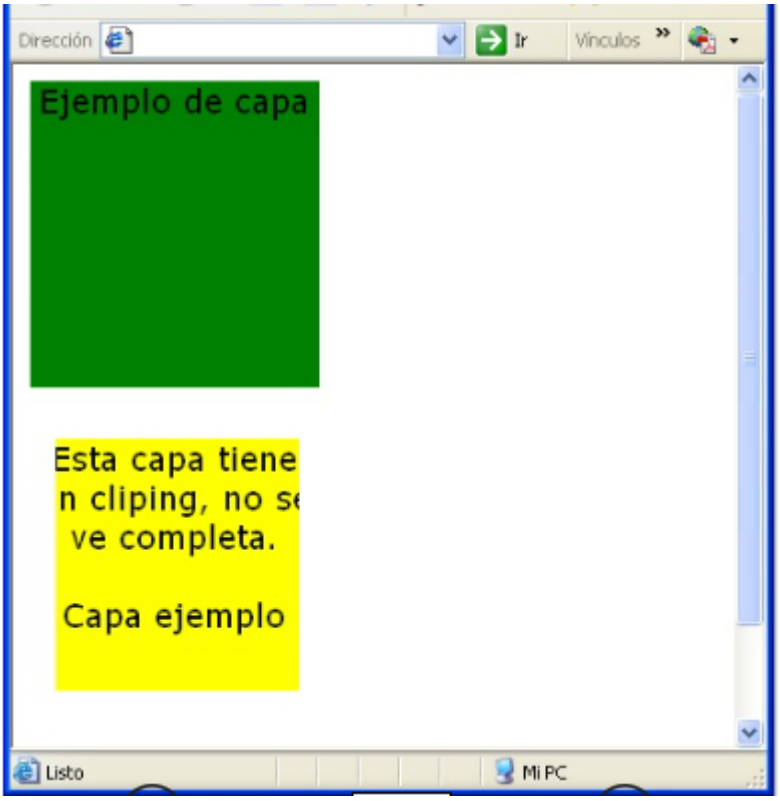
4.4. Ejemplos prácticos.

4.4.1. Efectos del atributo clip.

Este es un ejemplo de utilización de capas con el atributo clip, que define un área más pequeña que la propia capa y por eso no permite ver la totalidad del texto que está escrito dentro de dicha capa.

```
<html>
   <head>
      <title>Ejemplo con capas</title>
   </head>
   <body>
   <div style="height:180px; width:170px; left:10px; top:10px;
      position:absolute; visibility:visible; font-size:14pt;
```

```
      font-family:verdana; text-align:center; background-color:green">
      Ejemplo de capa
   </div>
   <div style="clip:rect(0,158,148,15); height:250px;
      width:170px; left:10px; top:220px; position:absolute;
      visibility:visible; font-size:14pt; font-family:verdana;
      text-align:center; background-color:yellow">
      Esta capa tiene un cliping, no se ve completa.
      <br><br>
      Capa ejemplo
   </div>
   </body>
</html>
```



Asocie cada elemento con su correspondiente:

- ***Position: Indica el tipo de posicionamiento de la capa***
- ***Clip: Atributo que tiene como objetivo recortar varias partes de la capa y que éstas no sean visibles.***
- ***Z-index: Sirve definir la altura que tendrá la capa en cuanto a la profundidad de la página Web***
- ***Top: Indica la distancia en vertical en la que se colará la capa.***

4.4.2. Mostrar/Ocultar un elemento.

Este ejemplo muestra al usuario dos capas denominadas Capa1 y Capa2, con un determinado formato mediante CSS, para después ocultarlas o mostrarlas mediante los eventos del ratón: onmouseover y onmouseout.

```
<html>
```

```
    <head>
       <title>Mostrar/Ocultar capas</title>
       <script>
          document.getElementById("Capa1").style.visibility="hidden"
       </script>
       <script language="Javascript">
          function visibilidad(nombre,valor)
          {
             document.getElementById(nombre).style.visibility=valor;
          }
    </script>
    </head>
    <body>
       <div id="Capa1" onmouseover="visibilidad('Capa2','visible');"
          onmouseout="visibilidad('Capa2','hidden');"
          style="position:absolute;
          width:60;
          height:20;
          top:15;
          top:15;
          left:15;
          background-color:red">
          Capa 1
       </div>
       <div id="Capa2"
          style="position:absolute;width:60; height:20;top:35;
          left:15; background-color:blue;visibility:hidden">
          Capa2</div>
    </body>
 </html>
```

El efecto será que al pasar el ratón sobre Capa1, ésta se muestra, mientras si el ratón está fuera del espacio de Capa1, la Capa2 permanecerá oculta.

Verdadero o falso

1. ***El parámetro position de las capas indica el tipo de posicionamiento de la capa***

   Verdadera

2. ***Mediante el parámetro Visibility se puede definir si la capa en cuestión será visible o, por el contrario, estará oculta al usuario Verdadera***
3. ***¿Definen las capas divisiones a las que se le pueden aplicar estilos de forma independiente al resto del documento? Verdadera***
4. ***Todos los atributos de las capas no se pueden modificar ya que cada capa funciona como parte independiente en una página web Falsa***
5. ***La etiqueta para definir capas < DIV > no es reconocida por Internet Explorer Falsa***
6. ***El atributo style de la etiqueta < DIV > se utiliza para poner un nombre a la capa Falsa***

Ideas clave

1. *Las capas son elementos de un documento HTML que pueden definir partes independientes en una página web y pueden aplicar estilos de forma independiente al resto del documento.*
2. *Las etiquetas que definen las capas son dos principalmente:*

```
<ilayer>...</ilayer> (esta etiqueta está obsoleta)
<div>...</div>
```

3. *Para poder colocar la capa en un lugar determinado dentro del documento HTML, se utiliza una serie de estilos siguiendo una nomenclatura utilizada para la hoja de estilos. De esta forma, se puede definir su posición y tamaño, si es visible o no e, incluso, se puede recortar la capa para hacer visible sólo una parte de la capa completa.*
4. *Los atributos que se pueden uitilizar para la etiqueta de capas*

   ...
   son los siguientes:
   - *Id: identifica la capa.*
   - *Style: define todos los estilos propios de la capa.*
5. *Existen varios parámetros de estilo propios de la capa:*
   - *Position: indica el tipo de posicionamiento de la capa.*
   - *Top: indica la distancia en vertical en la que se colocará la capa.*
   - *Left: indica la distancia horizontal respecto al lado derecho.*
   - *Height: indica la altura de la capa.*
   - *Width: indica la anchura de la capa.*
   - *Visibility: indica si la capa está visible o no.*
   - *Z-Index: indica la posición de la capa respecto al eje Z, es decir, define la altura que tendrá la capa en cuanto a la profundidad de la página web.*
   - *Clip: permite recortar varias partes de la capa y que éstas sean visibles.*
6. *Para poder acceder a los estilos de una capa y modificarlos dinámicamente se necesita utilizar el lenguaje Javascript en el DOM.*

5. XML

5.1. ¿Cómo nace XML?.

En la década de los 70, Charles Goldfarb, Ed Mosher y Ray Lorie inventaron GML para IBM. GML se utilizaba para describir una manera de maquetar documentos técnicos con etiquetas estructurales. Las iniciales vienen de los apellidos Goldfarb, Mosher y Lorie. Goldfarb inventó el término "mark-up languaje" para hacer un mejor uso de esas iniciales y así se convirtió en el Standard Generalised Markup Languaje.
SGML no es más que una especificación para definir lenguajes de marcas y a su vez es la madre de todos
los lenguajes de marcas como HTML, XML, XHTML, WML, etc. Pero mientras que SGML es muy efectivo pero complejo, HTML es muy sencillo pero limitado. Esta situación hizo surgir la

necesidad de un lenguaje que fuera tan efectivo como SGML y al mismo tiempo tan simple como HTML. Así, esta laguna fue cubierta por XML.
El desarrollo de XML comenzó en 1996 en Sun Microsystems con Jon Bosak y su equipo de trabajo en un proyecto para remodelar SGML. Aprovecharon lo mejor de SGML y produjeron algo simple pero con bastante poder. W3C también contribuyó a la creación y el desarrollo del estándar para XML. Así, las especificaciones de XML fueron resumidas en veintiseis páginas que, comparadas con las más de quinientas de SGML, resultaban mucho más sencillas de asimilar.

5.2. ¿Qué es XML?.

Un documento XML es un modelo de datos cuya sintaxis puede parecer similar al HTML, pero que tiene una característica fundamental que lo diferencia de éste: un documento XML describe el contenido de los elementos que etiqueta. Además el lenguaje XML separa el contenido del aspecto de una forma total.
Al describir un documento XML, las etiquetas pueden parecerse a las etiquetas HTML, con la diferencia de que las etiquetas en XML son etiquetas creadas por el propio programador definiendo en un documento público, o en el mismo documento XML, dichas etiquetas y los atributos que van a tener. Esta definición de las etiquetas de un documento XML es lo que se llama definición de tipo de documento: DTD (Document Type Definitions). En este DTD se definen las etiquetas, los atributos de cada una de ellas, el tipo de cada atributo y el contenido de las etiquetas.
Los elementos de un documento XML deben seguir una estructura jerárquica, manteniendo un elemento raíz único del que parten uno o varios de dichos elementos, los cuales siempre deben estar correctamente cerrados.

Por ejemplo:

```
<?xml version="1.0" encoding="UTF-8”?>
<!DOCTYPE LIBRO SYSTEM “libro.dtd">
<libreria>
<libro>
<titulo>El quijote</titulo>
<autor>Cervantes</autor>
</libro>
<libro>
<titulo>El buscón</titulo>
<autor>Quevedo</autor>
</libro>
</libreria>
```

En este ejemplo se observa la estructura jerárquica: el elemento raíz es “librería” y tiene varios elementos “libro” cada uno de los cuales, a su vez, tiene un elemento “titulo” y un elemento “autor”. Sin entrar en detalles, la primera línea especifica la versión de xml utilizada, y la segunda línea indica qué DTD se está usando para este documento, es decir, dónde están especificados los elementos que se utilizan en el documento XML.

Como etiquetas se puede utilizar cualquiera, podemos inventarlas, pero manteniendo una serie de requisitos que son principalmente dos:

- *Los caracteres a utilizar serían todas las letras y números, guiones, rayas y caracteres de punto, pero*
  ninguno más. Además una etiqueta sólo puede empezar por una letra.

- *La etiqueta no puede empezar por las letras xml o cualquier variación en mayúsculas y minúsculas de*
  XML.
- *XML es sensible a las mayúsculas y minúsculas, como JavaScript.*

Una etiqueta puede estar vacía pero, a diferencia del HTML, una etiqueta en XML que esté vacía debe escribirse y no (como se haría en el lenguaje HTML). Además las etiquetas pueden tener atributos, al igual que en el lenguaje HTML.
Hasta ahora se ha visto cómo se conforma un documento XML, que no es más que un modelo de datos. Para poder visualizarlo y darle un formato determinado a ese documento HTML, aparece el lenguaje XSL (Extended Style Language). Con este lenguaje se pueden realizar diferentes hojas de estilo que muestran el mismo documento XML en diferentes formatos: HTML, PDF, RTF, etc.

5.3. Sintaxis de XML.

Es aconsejable que los documentos XML comiencen por una declaración que describa la versión de
XML y el tipo de documento. Por ejemplo:

```
<?xml versión="1.0" encoding="UTF-8" ?>
```

La segunda línea es la denominada "declaración de tipo de documento". Se utiliza para definir el tipo
de documento que estamos diseñando. Lo que hacemos es definir qué DTD (Declaración de Tipo de
Documento) cumple y define el contenido del documento. Por ejemplo:

```
<?xml versión="1.0" encoding="UTF-8 ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XML 1.0
Strict//EN"
"DTD/xhtml1-strict.dtd">
<html> ... </html>
```

O bien:

```
<?xml version="1.0"
encoding="UTF-8" ?>
<?DOCTYPE ejemplo SYSTEM
http://...>
<nada> Ejemplo </nada>
```

Para que un documento XML sea válido tiene que tener asociado un DTD y además debe cumplir las
restricciones que la DTD expresa. Es imprescindible que solamente haya un elemento raíz en el cual
está contenido el resto de los elementos.

- *Documento Standalone: el documento Standalone controla los componentes de la DTD de tal forma que determina si son necesarios en el procesamiento del documento. El Standalone iría incluido en la declaración. Ejemplo:*

```
<?xml versión="1.0"
standalone="yes"?>
<nombre>Ana</nombre>
```

Standalone="yes" significa que no hay declaraciones de marcas externas a la entidad del documento.
Si por el contrario indicáramos "no" querría decir que existen dichas declaraciones de marcas. Por
tanto, la declaración de un documento standalone sólo indica la presencia de declaraciones externas.

- *Elementos: los elementos XML pueden tener contenido o estar vacíos. Un ejemplo de elemento con contenido sería:*

```
...<nombre> Ana
</nombre>
A la hora de utilizar ciertos símbolos usaremos las entidades predefinidas:
...<nombre> Ana </nombre>
<aviso tipo="emergencia"
gravedad="mortal">
Que no cunda el pánico
</aviso>…
```

Al contrario que HTML, para que XML permita elementos sin contenido, la etiqueta debe ser de la forma: . Esta etiqueta puede llevar atributos. Ejemplo:

```
... <identificador DNI="41865458x"/>…
<?xml versión="1.0" encoding="UTF-8" ?>
<!DOCTYPE html PUBLIC
"-//W3C//DTD XHTML 1.0 Strict// EN"
"DTD/xhtml1-strict.dtd">
<html>
<body bgcolor="red"><hr/>
<p>esto es un párrafo entre líneas</p><hr/></body>
</html>
```

---

esto es un párrafo entre
líneas

5.4. Validación de XML.

La validación de XML consiste en asegurarse de que el documento está bien formado y se ajusta a una estructura definida conforme al DTD. Para ello hay que seguir el DTD sin romper ninguna de las reglas que lo constituyen y prestar especial atención a cada una de ellas.
Los errores son el principal problema de los desarrolladores, pero si se tiene cuidado cuando se escribe el código, siguiendo el DTD, podemos evitarlos. Los elementos deben anidarse correctamente, ya que los elementos que se solapan causan errores. Si encontramos que el DTD

es demasiado estricto para el documento que queremos escribir, entonces es mejor modificarlo. Los errores pueden arruinar todo el trabajo y requieren una gran dedicación para solucionarlos, por tanto, muchos desarrolladores crean los documentos en editores de XML capaces de validarlos y que, además, están diseñados para evitar estos errores no dejando al autor que los introduzca.
Una manera más precisa y mejor de comprobar nuestros documentos y validarlos es utilizar un "parseador de código fuente", que consiste en un "analizador de sintaxis" que encuentra errores. Pueden ser muy útiles cuando no localizamos la razón por la cual el documento no funciona bien. Además, constituye una buena herramienta para ir comprobando el código a medida que se va escribiendo.
En la siguiente página web se puede introducir el código para comprobarlo online: https://www.magicparser.com/demo

5.5. Atributos y comentarios en XML.

Al igual que en HTML, los atributos son los encargados de definir las propiedades del documento. Por ejemplo:

```
… <alumno nota="9" asistencia="buena"> Ana
</alumno>
```

En la DTD se especifican los atributos que puede tener cada tipo de elemento, sus valores y el tipo de valor. Dichos atributos deben estar delimitados con comillas dobles o simples.

```
<noticia titulo="El código Da Vinci" autor="Dam Brown">
…
</noticia>
```

Es posible que un elemento con contenido pueda modelarse como un elemento vacío con atributos.

```
<perro><nombre>Willy</nombre><raza>Grifón</raza>
</perro> …
<perro raza="Grifón">Willy</perro>
…
<perro raza="Grifón" nombre="Willy"/> …
```

Pueden surgir problemas a la hora de dar valor a los atributos puesto que no son fácilmente ampliables y no pueden contener múltiples valores. Tampoco son fáciles de comprobar con un DTD y además, no se puede describir su estructura. Aquí ponemos un ejemplo que demuestra cómo los elementos pueden ser utilizados en lugar de los atributos. Los tres documentos contienen exactamente la misma información. El atributo "date" se utiliza en el primer ejemplo, el elemento "date" en el segundo y un elemento "date ampliado" en el tercero:

```
<?xml version="1.0" encoding="iso-8859-1”?>
<e-mail date="24/08/09">
<to>Thomas</to>
<from>Ana</from>
<subject>sorpresa....</subject>
<body>Prepárate para lo que viene
</body>
</e-mail>
```

Este primer documento contiene “date” como un atributo que no puede ser ampliado. Pero “date” como elemento en el segundo documento es más flexible:

```
<?xml version="1.0" encoding="iso-8859-1”?>
<e-mail>
<date="24/08/09">
<to>Thomas</to>
<from>Ana</from>
<subject>sorpresa....</subject>
<body>Prepárate para lo que viene</body>
</e-mail>
```

En este segundo documento “date” aparece como elemento y es mucho más ampliable:

```
<?xml version="1.0" encoding="iso-8859-1”?>
<e-mail >
<date><day>24</day>
<month>08</month>
<year>09</year></date>
<to>thomas</to>
<from>ana</from>
<subject>sorpresa....</subject>
<body>Prepárate para lo que viene</body>
</e-mail>
```

Los comentarios en XML tienen el mismo formato que en HTML y pueden estar incluidos en cualquier parte del documento. La cadena “— ” no puede aparecer en un comentario. Ejemplo:

```
… <!--Que gran perro es Willy -->
<perro raza="Grifón" nombre="Willy"/> …
```

5.6. Secciones CDATA e identificación del lenguaje.

Las secciones CDATA permiten especificar datos de tal forma que no se interprete como una marca XML. De esta forma se puede leer mejor un documento XML sin necesidad de descifrar los códigos de las entidades.
Las secciones CDATA comienzan de la siguiente forma:

```
“<![CDATA[“. Terminan con la cadena
“]]>”.
```

Tan sólo esta última cadena es reconocida como marca. Ejemplo:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes”?>
<ejemplos>
<descripción>
Esto es un ejemplo de HTML
</descripción>
<ejemplo><![CDATA[
<html><head><title> Mi pagina </title> </head>
] ]>
</ejemplo>
</ejemplos>
```

En la utilización de XML nos puede ser útil la especificación entre el lenguaje natural o el lenguaje formal. El atributo xml:lang sirve para especificar el lenguaje utilizado en el contenido del documento y los valores de dicho atributo de cualquier elemento del documento.

5.7. XML en el servidor.

Los documentos XML son archivos de texto plano como los de HTML. Por tanto, son fácilmente almacenables y generados por un servidor web estándar. Para almacenar los archivos XML en un servidor haremos exactamente igual que con los archivos HTML.
Abrimos un editor de código o un editor de texto y escribimos, por ejemplo, las siguientes líneas:

```
<?xml version="1.0" encoding="ISO-8859-1”?><note>
<from>Ana</from>
<to>Thomas</to>
<message>Prepárate para lo que sigue</message></note>
```

Guardamos el documento en el servidor con un nombre apropiado como, por ejemplo, “note.xml”.

Para generar XML con ASP desde el servidor, escribiremos un código y lo guardamos como un archivo ASP en el servidor web:

```
<%
response.ContentType=”text/xml”
response.Write(“<?xml version=’1.0’
encoding=’ISO-8859-1’?>”)
response.Write(“<note>”)
response.Write(“<from>Ana</from>”)
response.Write(“<to>Thomas</to>”)
response.Write(“<message>Prepárate para lo que sigue
</message>”)
 response.Write(“</note>”)
 %>
```

Como se puede comprobar, el valor de ContentType debe ser “text/xml”. El archivo ASP será devuelto por el servidor.
Para generar XML con PHP, se puede utilizar el siguiente código:

```
<?php
header("Content-type: text/xml");
echo "<?xml version='1.0' encoding='ISO-8859-1'?>";
echo "<note>";
echo "<from>Ana</from>";
echo "<to>Thomas</to>";
echo "<message>Prepárate para lo que sigue</message>";
echo "</note>";
?>
```

Este ejemplo utiliza ASP con ADO.
Para transformar XML con XSLT en el servidor se puede hacer lo siguiente, que transforma un archivo XML a XHTML en el servidor:

```
<%
'Load XML
set xml = Server.CreateObject("Microsoft.XMLDOM")
xml.async = false
xml.load(Server.MapPath("simple.xml"))
'Load XSL
set xsl = Server.CreateObject("Microsoft.XMLDOM")
xsl.async = false.
xsl.load(Server.MapPath("simple.xsl"))
0 'Transform file
 Response.Write(xml.transformNode(xsl))
 %>
```

El primer bloque de código crea una instancia del editor Microsoft XML (XMLDOM), y carga el archivo XML en memoria. El segundo bloque de código crea otra instancia para el editor y carga el archivo XSL en memoria. La última línea de código transforma el documento XML utilizando el archivo XSL y envía el resultado como XHTML al servidor.

Para guardar XML a un archivo en el servidor utilizando ASP podemos hacer lo siguiente:

```
<%
text="<note>"
text=text & "<to>Thomas</to>"
text=text & "<from>Ana</from>"
text=text & "<heading>Recordatorio</heading>"
text=text & "<body>Prepárate para lo que sigue</body>"
text=text & "</note>"
set xmlDoc=Server.CreateObject("Microsoft.XMLDOM")
xmlDoc.async="false"
xmlDoc.loadXML(text)
xmlDoc.Save("test.xml")
%>
```

5.8. ¿XML es el sustituto de HTML?.

Realmente el lenguaje XML es sustancialmente distinto al HTML. En un período de corto o medio plazo es imposible la total sustitución del HTML debido a la gran popularidad que tiene y la gran cantidad de páginas web que están realizadas en este lenguaje. A largo plazo, la mayor potencialidad de este lenguaje puede hacer que se extienda a numerosas páginas web dentro de

Internet, pero muchas otras páginas no necesitarán tal potencia y se preferirá mantener y crear páginas web con el lenguaje HTML por su
sencillez y rapidez a la hora de crear las páginas.
Por todo esto, la respuesta más lógica a la pregunta formulada en el título del apartado: ¿Es XML el sustituto de HTML?, sería: XML no es un sustituto del lenguaje HTML, sino una alternativa para conseguir mayor flexibilidad y potencia en las páginas web.

Test de altenativa simple/múltiple

1. *Una de las diferencias entre HTML y XML es:*
   XML describe el contenido de las etiquetas
2. *¿Qué nombre recibe el documento que define las etiquetas de un documento XML?*
   DTD (Document type definitions)
3. *¿Cuál de los siguientes requisitos debe darse en un documento XML?*
   La etiqueta no puede empezar por las letras xml o cualquier variación en mayúsculas y minúsculas de XML
4. *¿Para qué sirve el documento Standalone?*
   Determina si los componentes de la DTD son necesarios o no
5. *Los comentarios en XML:*
   Tienen el mismo formato que HTML y pueden incluirse en cualquier parte del documento

Ideas clave

1. *El éxito de Internet ha exigido una mayor vistosidad y funcionalidad en las páginas web realizadas en HTML.*
   Como consecuencia se ha empezado a desarrollar un nuevo lenguaje (un metalenguaje) denominado XML (Extensible Markup Language) que aproveche las ventajas del HTML pero, a su vez, permite ampliar su campo de aplicación.
2. *XML no es un sustituto del lenguaje HTML, sino una alternativa para conseguir mayor flexibilidad y potencia en las páginas web.*
3. *Un documento XML es un modelo de datos cuya sintaxis puede parecer similar al HTML, pero que tiene una característica fundamental que lo diferencia de éste: un documento XML describe el contenido de los elementos que etiqueta.*
4. *Las etiquetas en XML son etiquetas creadas por el propio programador definiendo en un documento público, o en el mismo documento XML, dichas etiquetas y los atributos que van a tener.*
5. *Esta definición de las etiquetas de un documento XML es lo que se llama definición de tipo de documento: DTD (Document Type Definitions). En este DTD se definen las etiquetas, los atributos de cada una de ellas, el tipo de cada atributo y el contenido de las etiquetas.*
6. *Los elementos de un documento XML deben seguir una estructura jerárquica, manteniendo un elemento raíz único del que parten uno o varios de dichos elementos, los cuales siempre deben estar correctamente cerrados.*
7. *En la utilización de las etiquetas se debe cumplir dos requisitos:*
   - *Los caracteres a utilizar serían todas las letras y números, guiones, rayas y caracteres de punto, pero ninguno más. Además una etiqueta sólo puede empezar por una letra.*

- *La etiqueta no puede empezar por las letras xml o cualquier variación en mayúsculas y minúsculas de XML.*

8. *Estas etiquetas pueden tener atributos, al igual que en el lenguaje HTML.*
9. *Para poder visualizar el documento XML en diferentes formatos aparece el lenguaje XSL (Extended Style Language), que permite realizar diferentes hojas de estilo que muestran el mismo documento XML en diversos formatos: HTML, PDF, RTF, etc.*

6.Usabilidad y accesibilidad

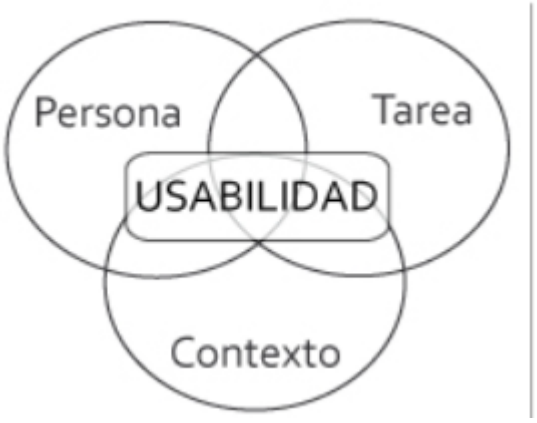
6.1. Usabilidad.

6.1.1. ¿Qué es la usabilidad?.

Intuitivamente se puede definir la usabilidad de un producto como "lo fácil de usar que es ese producto".

La ISO 9126-1 define la usabilidad como "la capacidad de un producto software de ser comprendido, aprendido, usado y de ser atractivo para el usuario, en condiciones específicas de uso".
Un diseñador web trabaja para seres humanos que desean realizar una determinada tarea: comprar un libro sobre informática, ver información sobre el tiempo, el tráfico, etc, en un entorno gráfico: la web. La usabilidad ayuda a que esta tarea se realice de una forma sencilla y eficaz.

El estándar introduce también otro término, la calidad en el uso, es decir, la capacidad del producto de software (o página web) para que determinados usuarios alcancen sus objetivos con efectividad, productividad, seguridad y satisfacción en un contexto específico de uso.
El principal objetivo es alcanzar la calidad en el uso para todo tipo de usuarios. La funcionalidad, fiabilidad, eficiencia y usabilidad determinan esta calidad para el usuario final.



6.1.2. Importancia de la usabilidad.

Un sitio web usable requerirá de un menor esfuerzo para su aprendizaje, y éste permanecerá mucho más tiempo en la memoria.
La usabilidad reduce los errores cometidos por los usuarios lo que lleva a que éstos realicen su tarea de una manera más eficiente y efectiva, aumentando así su satisfacción y mejorando su experiencia global con el sitio. Ante dos sitios web que ofrezcan productos o servicios similares los usuarios optarán por aquel que sea más fácil de aprender, eficiente en su uso, efectivo en los resultados y satisfactorio en la experiencia.

La reducción de errores además de provocar la satisfacción de los usuarios hace que los sitios sean más competitivos.
Ejemplo
Un sitio de comercio electrónico con una tasa de error muy baja hace que se incremente el número de ventas correctamente finalizadas, sin aumento de los costes. Los usuarios serán más fieles y mejorará la imagen de la empresa.

Un portal informativo con un diseño usable hará que los visitantes encuentren más rápidamente la información, con el correspondiente ahorro de tiempo, cosa que éstos agradecerán repitiendo sus visitas y recomendando el sitio.

¿Sabías que...
Usabilidad = Simplicidad = Satisfacción del usuario = Beneficios

Por todo esto, se hace imprescindible invertir en usabilidad, ya que ésta determina la decisión de una compra más que el precio del producto. Además, diseños usables implican diseños más simples y éstos son más baratos de construir.

6.1.3. Metodología para la usabilidad.

La usabilidad es uno de los parámetros más importantes a la hora de evaluar la calidad de un sitio web, ya que determina el grado de satisfacción del usuario respecto del sitio, de ello depende que sea utilizado o no. La norma ISO 8042 define la calidad como "El conjunto de propiedades y características de un producto o servicio, que le confieren aptitud para satisfacer unas necesidades explícitas o implícitas".
El principal problema de la usabilidad radica en que se trata de un parámetro demasiado abstracto para ser medida directamente.
J.Nielsen (el padre de la usabilidad) la descompone en cinco atributos para ser estudiada:

1. *Facilidad de retentiva:*
   Proporcionar al visitante la capacidad de regresar a la página y recordar cómo se navega en ella aún sin haberla visitado en un tiempo.
2. *Sencillez de aprendizaje:*

   La página debe ser sencilla en cuanto a que no presente dificultades en su navegación.
3. *Eficiencia en el uso:*

   Hacer que el visitante pierda el tiempo buscando un enlace que no existe, pulsando otro que no funciona o esperando a que se cargue algún elemento multimedia que no aporta nada puede hacer que éste abandone la página y busque otra que le reporte más utilidad
4. *Prevención de errores y recuperación:*

   Cuando el sistema presenta algún mensaje de error, éste debe ofrecer la información suficiente al visitante para que pueda seguir con su tarea. Mejor todavía, el sistema debe ayudar a prevenir errores.
5. *Satisfacción del visitante:*

   La impresión general del usuario ha de reflejarse en el sentimiento de que la página es agradable de usar.

¿Cómo se traducen estos requisitos en la estructura y resultado de la página? Los siguientes consejos
o ejemplos ayudan a dotar de usabilidad al sitio:

- *La importancia del buscador:*
  Muchos sitios web centran su usabilidad en la navegación y sorprende el hecho de que no hayan implementado un buen buscador. La idea básica de centrarse en la simple navegación y no colocar un
  buscador es asumir de antemano que la intención de los usuarios es navegar, lo cual es mucho asumir. Si no podemos ayudar a nuestros usuarios a acceder fácilmente a un producto que están buscando, ¿cómo podemos esperar que lo compre?
  Además, ¿es capaz nuestra página de filtrar por categorías? Muchos usuarios principiantes no se fían
  mucho de los buscadores y necesitan ayuda a la hora de introducir palabras clave. Ahí es donde una
  buena estructura de navegación puede darnos ventaja, además, puede convertirse en una agradable
  experiencia de exploración, especialmente para aquellos usuarios que no tienen una idea clara de lo
  que quieren comprar.
- *Registro de usuarios:*
  Algo que debemos preguntarnos es ¿realmente necesitamos un registro de usuarios? Un registro de
  usuarios es procedimiento por el cual un usuario introduce una serie de datos en un formulario (como
  mínimo su nick y su contraseña), pasando así a formar parte de una base de datos y pudiendo acceder
  a secciones que no se podrían visualizar sin dicho registro.
  Si nos fuera necesario dotar a la página de un registro, éste debe estar situado al comienzo de la experiencia de navegación y de tal manera que no "irrite" demasiado al usuario pidiéndole más datos de la cuenta.
- *El carrito de la compra:*
  En el caso en el que la página web sea una tiena online ¿nos deja el carrito de la compra modificar número de productos, eliminar o añadir ítems, etc., en cualquier momento de la navegación? ¿Qué ocurre si queremos guardar el estado de la compra y volver más tarde? ¿Continuará nuestra compra en el carrito tal y como la dejamos? Todas estas prestaciones deberían ser consideradas como necesarias en esta sección.
- *Precios:*
  Muchas páginas, por ejemplo las que venden billetes de avión, colocan el precio por persona aun habiendo indicado que el número total de pasajeros es más de uno. No muestran el precio total de la
  compra hasta que toca pagar. Pues bien, se dan casos de abandono de la compra por este motivo,
  pero es decisión del empresario mostrar los precios de una u otra forma.
- *Resolución de dudas:*
  Si al usuario le surgiera una duda ¿cómo podemos resolvérsela en ese momento? Existen múltiples
  formas de hacerlo, desde un servicio de chat hasta una sección de ayuda con explicaciones que den

respuesta a preguntas frecuentes. Eso sí, debemos asegurarnos de tener una solución antes de que
abandonen la compra por un momento de confusión.

- *Descripción del producto:*
  Si estuviéramos vendiendo productos directamente desde nuestra tienda online, ¿quién escribiría la
  descripción del producto, el informático o alguien de marketing? El informático hablaría con un lenguaje muy técnico mientras que el personal de marketing añadiría muchos eslóganes sin información relevante. Lo mejor es encontrar un equilibro entre los dos modos de describir el producto.
- *Orientación:*
  Es frustrante para un usuario cuando intenta comprar algo online no saber dónde está ni cuántos pasos más se requieren antes de finalizar la transacción. Es por eso que es importante dejarles saber en qué paso se encuentran y cuánto les falta para terminar. Esto se consigue con elementos simples
  como una fracción (paso 1 de 4, ó bien, 1/4).
- *Multimedia.*
  Uso de imágenes, sonidos o animaciones no simplemente para decorar la página, sino para aprovechar sus posibilidades en videos demostrativos, visitas guiadas, etc.
- *Cada página tiene sus pormenores, así que lo mejor para dotar de usabilidad a la página es ponerse del lado del usuario, indagar en ella y encontrar aquellos elementos susceptibles de ser mejorados con el fin de hacer más sencillos los pasos de la experiencia de compra.*
- *Ejemplos que NO mejoran la usabilidad en la web:*
  - *Descripción de la actividad del portal confusa o resumen ambiguo.*
  - *Imágenes muy pequeñas de grandes imágenes.*
  - *Formularios excesivamente restrictivos.*

6.2. Accesibilidad.

6.2.1. ¿Qué es y a quién va dirigida la accesibilidad web?.

Se puede definir la accesibilidad web como "la posibilidad de que un producto o servicio web pueda ser accedido y usado por el mayor número posible de personas, independientemente de las limitaciones propias del individuo (discapacidades, idioma, conocimientos, experiencia,...) o de las derivadas del contexto de uso".
La accesibilidad debe ser entendida como "parte de", y al mismo tiempo "requisito para", la usabilidad.
El acceso a las páginas web y a su contenido presenta una serie de dificultades para las personas con discapacidades físicas, sensoriales o cognitivas. Estas difi cultades se podrán reducir considerablemente si los desarrolladores web tienen en cuenta las necesidades de las personas con discapacidad y la diversidad de formas de acceso (condicionada por los distintos terminales existentes, el software, la velocidad de la conexión, etc...) y respetan unas normas en cuanto a la estructura y diseño de las páginas web.

No todo el mundo utiliza para navegar por internet los mismos dispositivos, ni todos los usuarios navegan de la misma forma:

- *Las personas ciegas suelen utilizar un programa lector de pantallas para acceder al contenido del navegador. Mediante herramientas de síntesis de voz, escuchan el contenido textual de las páginas, o lo leen en Braille a través de un dispositivo especial.*
- *Los usuarios con deficiencia visual suelen utilizar un magnificador de pantalla para ampliar la imagen, activan el mayor tamaño de fuente disponible o suelen mostrar la página con el mayor contraste posible entre el fondo y el contenido desactivando los colores predefinidos en las páginas.*
- *Las personas sordas o con deficiencia auditiva no perciben avisos sonoros ni pueden acceder a la banda de audio de los elementos multimedia. La subtitulación adaptada y la información contenida en imágenes y diagramas son de gran utilidad para este tipo de usuarios, así como la posibilidad de incluir videos en lengua de signos.*
- *Determinadas deficiencias motrices pueden impedir manejar el ratón. Estos usuarios controlan el ordenador desde el teclado o desde dispositivos especiales como los pulsadores, usando las ayudas de accesibilidad de las que disponga su sistema operativo.*
- *Las personas con dificultades cognitivas leves pueden tener problemas para interpretar adecuadamente el lenguaje simbólico, como los iconos, y pueden perderse si la estructura de la web es compleja. Un vocabulario sencillo y una sintaxis simple son elementos fundamentales para que estos usuarios comprendan adecuadamente los textos.*

Por tanto, los desarrolladores de sitios web deben tener en cuenta esta variedad de situaciones y procurar que los sitios que construyan sean accesibles para todo tipo de usuarios.

6.2.2. ¿Por qué un sitio web debe ser accesible?.

Un sitio web debe ser accesible por los siguientes motivos:

1. *Para acceder de forma más cómoda a los visitantes (usuarios potenciales) de un sitio web.*
2. *Para tomar una posición de liderazgo en el sector de negocio en el que se incluya el sitio web.*
3. *Para ganarse la fidelidad de los usuarios del sitio.*
4. *Para no realizar ningún tipo de discriminación ante posibles usuarios.*
5. *Simplemente por un buen sentido comercial, al incrementar el número potencial de clientes.*
6. *Para los sitios web institucionales la justificación es simple, facilitar la información a la sociedad.*

Cuestiones legales y normativas: Ley 51/2003, de 2 de diciembre, de igualdad de oportunidades, no discriminación y accesibilidad universal de las personas con discapacidad. Norma AENOR 139.802: "Informática para la salud. Aplicaciones informáticas para personas con discapacidad. Requisitos de accesibilidad de las plataformas informáticas. Soporte lógico". Resolución del consejo Europeo sobre "Accesibilidad electrónica"- Mejorar el acceso de personas con discapacidad a la sociedad del conocimiento: Resolución del 14 enero del 2003.

6.2.3. Recomendaciones WAI.

El W3C (Consorcio World Wide Web) tiene como misión guiar la web hacia su máximo potencial a través del desarrollo de protocolos y pautas que aseguren el crecimiento futuro de la web. Este consorcio ha publicado una serie de recomendaciones que explican pautas a seguir a la hora de diseñar un sitio web, utilizar una herramienta de edición o simplemente utilizar un navegador de internet. A esto se le denomina WAI (Iniciativa de accesibilidad web). Debido a la gran cantidad de usuarios que acceden a la red, se pueden dividir las recomendaciones WAI en las categorías siguientes:
WCAG (Recomendaciones de accesibilidad para el contenido web):
Este tipo de recomendaciones se centran en cómo se deben de organizar los contenidos de la web. Están pensadas para:

- ***Desarrolladores de contenido web (páginas web, sitios web,...).***
- ***Desarrolladores de herramientas de autor para la web.***
- ***Desarrolladores de herramientas de evaluación de accesibilidad web.***

Se tratan, en definitiva, de las normas que todo diseñador debe de seguir para crear una web accesible y de calidad, disponible para todos los usuarios independientemente de las capacidades, limitaciones o dispositivos de acceso que empleen.

ATAG (Recomendaciones de accesibilidad para herramientas de edición):
Estas normas están dirigidas a los desarrolladores de herramientas de edición web (HTML, XML, WYSIWYG), e indican cómo conseguir que dichas herramientas generen código HTML accesible y como desarrollar una Interfaz de usuario accesible.
UAAG (Recomendaciones de Accesibilidad para Agentes de Usuario):
Son las normas que afectan a los desarrolladores de navegadores web, principalmente, y marcan las pautas para que los interfaces gráficos empleados sean accesibles y para que el navegador contenga ayudas a la navegación.

6.2.4. ¿Qué son los Niveles WAI?.

Las recomendaciones de accesibilidad para la web (WCAG) de dividen a su vez en tres niveles: A, AA y AAA.
Con el fin de hacer accesibles las páginas a todo tipo de usuarios, se han establecido ciertas recomendaciones que deberían seguir los diseñadores con el fin de facilitar el acceso a todos los usuarios que puedan visitar el sitio web. Para ello se han establecido tres prioridades como puntos de verificación, para facilitar a los diseñadores la adaptabilidad y saber qué tipo de usuarios tendrán mayor difi cultad al visitar dicho sitio.

- ***Prioridad 1.***

  Se trata de la prioridad más baja y representa los requisitos básicos. En el caso de que el desarrollador web no los cumpla, ciertos grupos de usuarios no podrán acceder a la información del sitio web.
  Por ejemplo: para facilitar que personas o dispositivos que no puedan visualizar gráficos, obtengan
  una representación alternativa textual.

- ***Prioridad 2.***

  Esta prioridad va a permitir eliminar muchas barreras y aunque el usuario pueda acceder al contenido, es posible que encuentre muchas dificultades para llevarlo a cabo.

- ***Prioridad 3.***

  Se trata de la prioridad más alta y mejorará considerablemente la accesibilidad. En caso de no cumplirse, algunos usuarios tendrán ciertas dificultades para acceder al contenido.

En función de las prioridades que cumplan las páginas o sitios web, pueden presentar distintos niveles de conformidad. Estos son:
A: se satisfacen todos los puntos de verificación de prioridad 1.
AA: se satisfacen todos los puntos de verificación de prioridad 1 y 2.
AAA: se satisfacen todos los puntos de verificación de prioridad 1, 2 y 3.

Si el sitio web se ajusta al nivel de recomendaciones WAI requerido, figurarán en el los iconos siguientes, en función del nivel de conformidad al que se ajusten.

¿Cuáles de los siguientes ejemplos mejorarán la usabilidad en la web?

- Usar imágenes con el objetivo de decorar la página web
- Navegación rápida por el sitio
- Formularios muy restrictivos
- Formato simple de navegación
- Incluir una caja de búsquedas interna

6.2.5. ¿Cómo construir con accesibilidad?.

Consideraciones a tener en cuenta basadas en las normas WAI, y que ningún diseñador de páginas web debería incumplir para crear sitios web accesibles.

- ***Proporcionar alternativas textuales para todo contenido no textual, de manera que pueda modificarse para ajustarse a las necesidades de las personas, como por ejemplo en una letra mayor, braille, voz, símbolos o un lenguaje más simple.***
- ***Proporcionar alternativas sincronizadas para contenidos multimedia sincronizados dependientes del tiempo.***
- ***Crear contenidos que puedan presentarse de diversas maneras (como por ejemplo una composición más simple) sin perder la información ni su estructura.***
- ***Hacer más fácil para los usuarios ver y oír el contenido, incluyendo la separación entre primer plano y fondo.***
- ***Hacer que toda funcionalidad esté disponible a través del teclado.***
- ***Proporcionar a los usuarios con discapacidades el tiempo suficiente para leer y usar un contenido.***
- ***No diseñar un contenido de manera que se sepa que puede causar ataques.***
- ***Proporcionar medios que sirvan de ayuda a los usuarios con discapacidades a la hora de navegar, localizar contenido y determinar dónde se encuentran.***
- ***Hacer el contenido textual legible y comprensible.***
- ***Crear páginas web cuya apariencia y operabilidad sean predecibles.***
- ***Ayudar a los usuarios a evitar y corregir errores. Maximizar la compatibilidad con agentes de usuario actuales y futuros, incluyendo tecnologías asistivas.***

Asocie las distintas recomendaciones y niveles WAI con las expresiones de la derecha:

1. ***WCAG: Recomendaciones de accesibilidad para el contenido web.***
2. ***ATAG: Dirigidas a los desarrolladores de herramientas de edición web.***
3. ***A: Satisfacen la prioridad más baja y representa los requisitos básicos.***
4. ***UAAG: Recomendaciones de accesibilidad para Agentes de Usuario.***
5. ***AA: Satisfacen los puntos de prioridad 1 y 2.***

6.2.6. Nuevas tendencias.

Además de los usuarios que necesitan ayuda citados en el tema, existen otros que necesitan aún más ayuda, se trata de personas con parálisis tan severas que no pueden mover la cabeza. Pues, a partir de ahora basta con mover un ojo para controlar un ordenador y a través de éste muchas cosas más (tareas domésticas, leer un libro, escuchar música, etc…. Esto es posible gracias al sistema Iriscom.
Para ello basta con que el paciente pueda mover uno de sus ojos, y frente a la pantalla del ordenador pueda guiar el puntero del ratón con su mirada y ejecutar acciones, como por ejemplo hacer clic parpadeando lentamente.

De las siguientes sentencias, ¿cuáles se han de seguir para diseñar un sitio web accesible?

- Proporcionar subtítulos y transcripción del sonido
- Proporcionar descripción del video
- No utilizar applets y plugins como contenido alternativo
- Usar el atributo ALT para describir la función de cada elemento visual
- Usar pinche aquí en los enlaces
- Utilizar el elemento noframes con sentido
- Uso del elemento map de tipo servidor
- Usar listas para la organización de las páginas

Ideas clave

1. *Intuitivamente se puede definir la usabilidad de un producto como "lo fácil de usar que es ese producto". La ISO 9126-1 define la usabilidad como "la capacidad de un producto software de ser comprendido, aprendido, usado y de ser atractivo para el usuario, en condiciones específicas de uso".*
2. *Usabilidad=Simplicidad=Satisfacción del usuario=Mayores Beneficios.*
3. *Uno de los parámetros más importantes para medir la calidad de un sitio web es la usabilidad.*
4. *Se puede definir la accesibilidad web como "la posibilidad de que un producto o servicio web pueda ser accedido y usado por el mayor número posible de personas, independientemente de las limitaciones propias del individuo (discapacidades, idioma, conocimientos, experiencia,...) o de las derivadas del contexto de uso".*
5. *La accesibilidad debe ser entendida como "parte de", y al mismo tiempo "requisito para", la usabilidad.*
6. *Los desarrolladores de sitios web deben tener en cuenta esta variedad de situaciones y procurar que los sitios que construyan sean accesibles para todo tipo de usuarios.*
7. *Un sitio web debe ser accesible por los siguientes motivos:*
    1. *Para acceder de forma más cómoda a los visitantes (usuarios potenciales) de un sitio web.*
    2. *Para tomar una posición de liderazgo en el sector de negocio en el que se incluya el sitio web.*
    3. *Para ganarse la fidelidad de los usuarios del sitio.*
    4. *Para no realizar ningún tipo de discriminación ante posibles usuarios.*
    5. *Simplemente por un buen sentido comercial, al incrementar el número potencial de clientes.*
    6. *Para los sitios web institucionales la justificación es simple, facilitar la información a la sociedad.*
    7. *Cuestiones legales o normativas.*

8. *El W3C (Consorcio World Wide Web) tiene como misión guiar la web hacia su máximo potencial a través del desarrollo de protocolos y pautas que aseguren el crecimiento futuro de la web. Este consorcio ha publicado una serie de recomendaciones que explican pautas a seguir a la hora de diseñar un sitio web, utilizar una herramienta de edición o simplemente utilizar un navegador de internet. A esto se le denomina WAI (Iniciativa de accesibilidad web).*
9. *Las recomendaciones WAI se pueden dividir en tres categorías:*
    1. *WCAG: Recomendaciones de accesibilidad para el contenido web.*
    2. *ATAG: Recomendaciones de accesibilidad para herramientas de edición.*
    3. *UAGG: Recomendaciones de accesibilidad para Agentes de Usuario.*
10. *Se han establecido tres prioridades como puntos de verificación, para facilitar a los diseñadores la adaptabilidad y saber qué tipo de usuarios tendrán mayor dificultad al visitar dicho sitio.*
    1. *Prioridad 1. Se trata de la prioridad más baja y representa los requisitos básicos. En el caso de que el desarrollador web no los cumpla, ciertos grupos de usuarios no podrán acceder a la información del sitio web.*
    2. *Prioridad 2. Esta prioridad va a permitir eliminar muchas barreras y aunque el usuario pueda acceder al contenido, es posible que encuentre muchas dificultades para llevarlo a cabo.*
    3. *Prioridad 3. Se trata de la prioridad más alta y mejorará considerablemente la accesibilidad. En caso de no cumplirse, algunos usuarios tendrán ciertas dificultades para acceder al contenido*
11. *En función de las prioridades que cumplan las páginas o sitios web, pueden presentar distintos niveles de conformidad:*
    1. *A: se satisfacen todos los puntos de verificación de prioridad 1.*
    2. *AA: se satisfacen todos los puntos de verificación de prioridad 1 y 2.*
    3. *AAA: se satisfacen todos los puntos de verificación de prioridad 1, 2 y 3.*

7. Desarrollo web avanzado

7.1. ¿Qué es una aplicación web?.

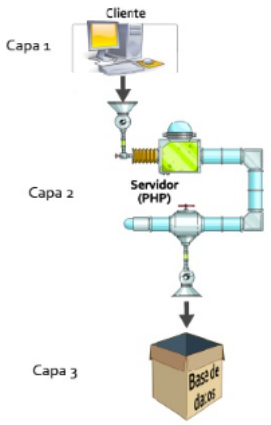
Hace años, una aplicación se basaba en la estructura cliente-servidor en la cual la aplicación consistía en un programa cliente más una interfaz de usuario, que tenían que ser instalados en cada una de las estaciones de trabajo desde las que se iba a acceder a la aplicación. Esto ocasionaba problemas, por ejemplo, de coste de soporte técnico, ya que cuando en el servidor se introducía una mejora, ésta tenía que ser instalada en cada uno de los clientes.
Actualmente, las aplicaciones web generan dinámicamente una serie de páginas, que se podrían identificar como la aplicación cliente y su interfaz, soportadas por navegadores web comunes mediante HTML. Tienen la ventaja de no tener que instalar una aplicación cliente en cada una de las estaciones de trabajo, además no es necesario actualizarlas cuando se introduzca una mejora en el servidor ahorrando así el coste de soporte técnico que conllevaba la estructura cliente-servidor. Para ello se utilizan lenguajes del lado del servidor consiguiendo así una

interfaz de usuario más interactiva, atractiva y eficiente.
Recuerda
Una aplicación web es "una aplicación informática que se utiliza accediendo al servidor a través de un sistema de red como puede ser Internet o una intranet.
Una ventaja significativa de las aplicaciones web es que, en lugar de crear clientes específicos para los distintos sistemas operativos (Windows, Mac OS X, GNU/Linux,...), la aplicación se escribe una vez y se muestra en cualquier navegador, pudiendo ser vista por cualquier sistema operativo que tenga instalado dicho navegador.

7.2. Estructura de una aplicación web.

Aunque existen más variaciones, la estructura más común de una aplicación web es la basada en tres capas:

- *Capa 1: está formada por el cliente o navegador y en ella se visualiza el trabajo final.*
- *Capa 2: la se sitúa normalmente en el servidor y en ella se llevan a cabo prácticamente todos los procesos. Utiliza para ello alguna tecnología web dinámica como: PHP, ASP, JSP,...*
- *Capa 3: normalmente es el sistema de administración de la base de datos. Es decir, donde los datos requeridos por la capa media son almacenados. Se sitúa en un servidor separado conocido como el servidor de base de datos.*

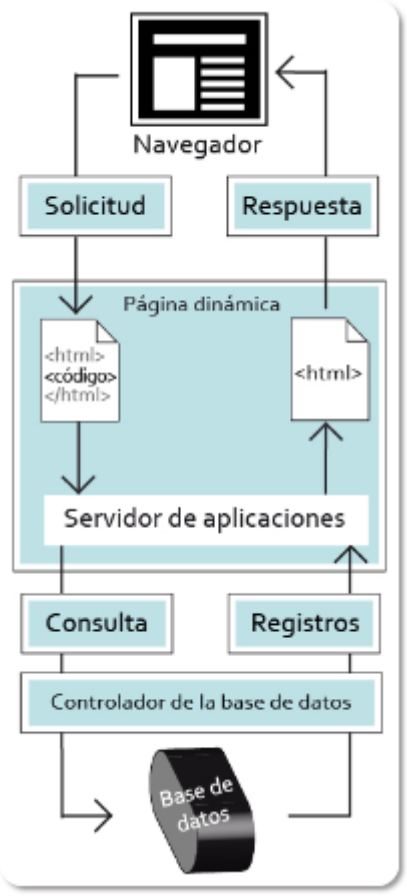


La forma de interactuar las tres capas es la siguiente:
El navegador web le realiza una petición a la capa intermedia usando HTML y algún lenguaje del lado
del cliente como JavaScript. La capa media traduce dichas peticiones a consultas SQL que son transmitidas a la base de datos. Ésta tras recibir la sentencia SQL, la ejecuta y devuelve el resultado
a la capa media, generando una página HTML con dicho resultado. La capa intermedia le envía al navegador la página HTML, que es capaz de entender y ejecutar sin problemas, permitiendo así su visión por parte del usuario.

- *Paso 1: El navegador web solicita la página dinámica.*
- *Paso 2: El servidor Web localiza la página y la envía al servidor de aplicaciones.*
- *Paso 3: El servidor busca instrucciones en la página.*
- *Paso 4: El servidor de aplicaciones envía la consulta al controlador de la base de datos.*
- *Paso 5: El controlador ejecuta la consulta en la base de datos.*
- *Paso 6: El juego de registros se devuelve al controlador.*
- *Paso 7: El controlador pasa el juego de regrstros al servidor de aplicaciones.*

- *Paso 8: El servidor de aplicaciones inserta los datos en una página y luego pasa la página al servidor Web.*
- *Paso 9: El servidor Web envía la página finalizada al navegador solicitante.*

No todas las aplicaciones hacen uso de todas las capas, por ejemplo una calculadora o un conversor de monedas, no utiliza la tercera capa, ya que no hace uso de datos almacenados. En las páginas web estáticas que se utilizan para mostrar publicidad solo se hace uso de la primera capa.



7.3. El lenguaje PHP.

El lenguaje PHP trata de ofrecer a los programadores la mejor relación entre rendimiento y flexibilidad, debido a que se trata de un lenguaje tanto intérprete como compilador. Una de sus ventajas más significativas es la rapidez y fluidez de sus scripts, de ahí que se trate de uno de los lenguajes del lado del servidor más usados actualmente.
Es un lenguaje muy apropiado para iniciarse en la programación web, puesto que todos los navegadores lo
soportan (multiplataforma), es de código abierto y se integra muy bien con sistemas gestores de bases de datos como MySQL.
La programación web tiene una serie de peculiaridades que se citan a continuación:

- *El interfaz de usuario es un navegador web, por lo tanto el programa PHP debe preparar la salida en*
  formato HTML (interpretado por todos los navegadores). Esto obliga a tener que mezclar en el mismo
  documento el código PHP con HTML, aunque también pueden intervenir otras tecnologías como CSS o
  JavaScript.

- *Las líneas de código PHP se delimitarán por los símbolos lo que acarrea un problema en los proyectos*
  grandes, ya que hace que los programas sean difícil de mantener. La solución consiste en utilizar técnicas como el patrón MVC (Model-View-Controller) y la programación por capas.

La gran funcionalidad de PHP radica en la amplia gama de librerías que contiene, permitiendo así la realización de multitud de tareas:

- *Funciones de correo electrónico:*
  Permite enviar un e-mail a una persona o lista pudiendo seleccionar distintas opciones como el e-mail de procedencia, asunto, persona a responder,...
- *Gestión de bases de datos:*

  Ofrece interfaces para acceder a la mayoría de las bases de datos existentes en el mercado, pudiendo editar el contenido del sitio sin problemas.
- *Gestión de archivos:*

  Contiene una amplia librería de funciones para la gestión de archivos, pudiendo realizar con ellos operaciones como crear, borrar, mover, modificar, etc. e incluso transferirlos mediante el protocolo FTP a partir de líneas de código.
- *Otras opciones permitidas son:*

  Tratamiento de imágenes (útil cuando se tienen gran cantidad de imágenes), crear botones dinámicos (se utiliza el mismo diseño y sólo se cambia el texto),... a través de la librería de funciones gráficas.

Para poder trabajar con el lenguaje PHP en modo local es necesario convertir el ordenador en un servidor, instalando cualquiera de los servidores compatibles con el sistema operativo que dicho ordenador tenga instalado. Posteriormente habrá que instalar PHP (disponible en la página oficial de php, recuerde que se trata de un software gratuito).
Como se ha comentado, la instalación del tipo de servidor depende del sistema operativo que el ordenador tenga instalado. Bajo plataformas UNIX/Linux la combinación más habitual es instalar Apache. Para entornos Windows 2000, XP Professional y server o NT, PHP puede instalarse junto al servidor IIS ó al servidor Apache. Es interesante instalar éste último ya que PHP está pensado para trabajar junto a él. Además está considerado como el servidor más popular debido a su arquitectura modular, software de código abierto (Open Source) y carácter multiplataforma.

Instalación del servidor web Apache en plataformas Windows

1. *Descargar el servidor de la página oficial Web de Apache*

   Descargar la versión para instalar, binary (Windows).
2. *Ejecutar el archivo descargado, Win 32-binary. Aparecerá un asistente, que guiará el proceso de la instalación. Éste solicitará una serie de datos como el nombre del dominio o el nombre del servidor (por ejemplo, "localhost").*
3. *Activar la opción: "que el servidor responda para todos los usuarios en el puerto 80", permitiendo al servidor comunicarse a través de dicho puerto.*
4. *Una vez instalado, aparecerá un icono en la barra de tareas de Windows indicando que el servidor está en marcha.*

5. *Para comprobar que Apache funciona correctamente, abrir el explorador e intentar acceder al servidor mediante la URL http://localhost.*
6. *Si todo ha ido bien, Apache mostrará un mensaje de bienvenida del tipo: "La instalación se ha completado sin errores" o "El proceso de instalación ha sido un éxito".*

Una vez que tiene instalado el servidor, el paso siguiente sería instalar PHP.

Instalación de PHP para trabajar con Apache bajo plataformas Windows:

Existen dos formas de instalar PHP, una es instalarlo como un CGI ó como módulo en Apache. Esta última opción es adecuada si la instalación se lleva a cabo en un servidor que se encuentra conectado a Internet 24 horas al día, y que trabaja para todo tipo de usuarios. Si lo que desea es trabajar con PHP a nivel local, como es el caso, lo más apropiado es instalarlo como un CGI, aunque se pierda en seguridad y potencia.
El proceso de instalación de PHP como un CGI, es el siguiente:

1. *Descargar el fichero ZIP "php.net" y descomprimirlo en la carpeta c: PHP (lo descomprime por defecto en esa ruta).*
2. *Dentro de la ruta c:PHP se encuentra el fichero php.ini-recomended, copiarlo en la carpeta c:Window y renombrarlo como php.ini.*
3. *Indicarle al servidor donde se encuentra instalado PHP. Para ello, editar el fichero "http.conf", situado en la carpeta conf, en la carpeta de instalación del Apache. Por defecto la ruta suele ser:*
   C:archivos de programaapache groupapache 2conf.
4. *Abrir el fichero y escribir al final:*
   ScriptAlias/php/ "c:/php/"
   AddType application/x-httpd-php.php
   Action aaplication/x-httpd-php "/php/php.exe"
   Donde se le indica el lugar en el que se encuentra el ejecutable de php, asociándolo a los ficheros .php que se encuentran dentro del servidor.
5. *Para que la instalación se realice de forma correcta, reiniciar el servidor Apache.*

Para probar PHP, es recomendable crear un fichero php con el contenido: y guardarlo en la extensión .php dentro de la carpeta de instalación de Apache. Ahora puede ejecutarlo a través del navegador, escribiendo en él la URL: http://localhost/info.php

Para obtener una página verdaderamente dinámica es necesario conectar los programas instalados con un gestor de bases de datos, por ejemplo MySQL.

Instalación del gestor de base de datos gratuito MySQL:

1. *Descargar la última versión del Win-32.zip, en la dirección: https://dev.mysql.com/*
2. *Descomprimir y ejecutar el archivo Win-32.zip. Aparecerá un asistente que le guiará en el proceso de instalación.*

3. *Al final de la instalación, se puede configurar MySQL a través de un asistente de una forma sencilla. El asistente solicitará que seleccione entre varias opciones, de ellas se han de marcar las siguientes:*
    1. *Utilizar una configuración estándar.*
    2. *Instalar MySQL como un servicio de Windows (Install as Windows services).*
    3. *Que se lance automáticamente (Launch the MySQL server automatically).*
    4. *Añadir el directorio que contiene los ejecutables de MySQL a la variable Path de Windows para que puedan correr desde cualquier lugar.*
    5. *Introducir una contraseña para el administrador de la base de datos (root)*
4. *Una vez se han seguido correctamente todos los pasos del asistente, pulsar execute para cargar la configuración se leccionada e iniciar MySQL.*

Ahora ya tenemos instalado todo el software necesario para trabajar en modo local con PHP.

¿Cómo funciona una página PHP?

El servidor reconoce la extensión del programa PHP: .php, .phtml, ..., e interpreta y ejecuta todo lo que hay entra las etiquetas PHP.
Envía al navegador el script ejecutado y transformado en HTML junto con el resto del código, asume que éste se encuentra escrito en HTML y por tanto es perfectamente interpretado por el cliente.
Como ya se ha comentado el código PHP se escribe junto al HTML, de ahí la importancia de utilizar correctamente los modos de abrir y cerrar las etiquetas PHP.
Ejemplos:

1. *Conexión y listado de una base de datos:*
   Este es el código necesario para hacer la conexión entre PHP y una base de datos MySQL.
   Con la instrucción mysql_connect se realiza la conexión con el servidor, los parámetros son:
   - *El nombre del servidor (obligatorio)*
   - *El nombre de usuario (obligatorio)*
   - *El password del usuario (si tiene)*

   Después se ha de conectar con la base de datos utilizando la instrucción:
   mysql_select_db con los parámetros:
   - *Nombre de la base de datos .*
   - *Nombre de la conexión con el servidor utilizado.*

```
<?
$conexio = mysql_connect("localhost","root","");
mysql_select_db ("bd_agenda", $conexio) OR die ("No se puede conectar");
$sql="SELECT * FROM agenda";
$result=mysql_query($sql,$conexio);
if ($row= mysql_fetch_array($result))
{
 echo "<TABLE BORDER='1'>";
 echo "<TR><TD>Nombre</TD><TD>Teléfono</TD></TR>";
```

```
DO
{
    echo "<TR><TD>".$row["nombre"]."
    </TD><TD>".$row["tel"]."</TD></TR>";
}
WHILE ($row=mysql_fetch_array($result));
    echo "</TABLE>";
}
?>
```

Como puede observarse la mezcla del lenguaje de servidor PHP con HTML genera un código que se puede complicar bastante de no separarlos correctamente.

Ver el número de usuarios que en cada momento están navegando en su sitio web.

```
<?
// Configuración de las variables
$arch = "dades.txt";
// Archivo de datos de los usuarios (coloque la ruta completa dentro del
servidor).
$logoutTime = 900;
// Tiempo tras el cual un usuario es considerado inactivo (en segundos).
$IP = $REMOTE_ADDR;
$count = 0;
$res = '';
$fp = fopen($arch,"r");
$t = time();
$content = fread($fp,filesize($arch));
fclose($fp);
$lineas = split("n",$content);
for ($i=0; $i < count($lineas); $i++) {
    $datos = split(':',$lineas[$i]);
    $n = $t - $logoutTime;
    if ($datos[0] != $IP && $datos[1] > ($n)) {
        $res .= "$datos[0]:$datos[1]n";
        $count++;
    }
}
$res .= "$IP:$tn";
$count++;
$fp = fopen($arch,"w");
fwrite ($fp,$res);
fclose($fp);
echo "Hay $count visitantes activos";
?>
```

Formulario para enviar correo con PHP desde la web.

```
<?
    header("Cache-Control: no-cache, must-revalidate");
?>
```

```
<html>
<body>
<?
    echo "<FORM METHOD="POST" ACTION=$PHP_SELF >";
    echo "Destinatario: <INPUT NAME=destinatario value = $destinatario><P>";
    echo "Texto del mail <TEXTAREA NAME=texto COLUMNS=20 ROWS=10 >";
    echo $texto;
    echo "</TEXTAREA><P>";
    echo "<INPUT TYPE=SUBMIT NAME=boton VALUE=Enviar>";
    echo "</FORM>";
    if ($boton=="Enviar")
        mail("$destinatario",$motivo, $texto,"FROM: info@bdat.comnX-Mailer:
PHP");
 ?>
 </body>
 </html>
```

7.3.2. El lenguaje JSP (JavaServer Pages).

Es un lenguaje basado en la tecnología Java. Por tanto, se puede ejecutar en diferentes servidores debido al carácter multiplataforma de Java. El código que define sus scripts se introduce mezclado con el HTML, delimitado por las etiquetas de apertura y cierre propias de los script de servidor en sintaxis Java, por lo que será interesante poseer conocimiento de programación en Java.
¿Cómo funciona una página JSP?
Las páginas JSP se desarrollan de una forma muy parecida al resto de programas de servidor y se ejecutan en una máquina virtual Java. Los archivos generados tienen extensión .jsp.
La primera vez que un cliente solicita una página JSP, se carga el programa más el intérprete, de manera que ya queden cargados para las siguientes peticiones, ahorrando así el tiempo de invocarlos. Esto le permite hacer una serie de cosas de forma más eficiente como puede ser la conexión a una base de datos y el manejo de sesiones. La máquina virtual que se ha cargado compila el programa JSP a un programa en Java y de éste se crea una clase que se ejecuta en el servidor como un servlet (programa que recibe peticiones y genera a partir de ellas una página web).
Los servlets y los JSP se sitúan en unos servidores específicos denominado contenedores de servlets. Ejemplos de contenedores de servlets son: Resin de Caucho Tecnologies, Bea Weblogic, Lutrus Ebhydra, y el más popular Tomcat del consorcio Apache.
Para poder crear una página JSP, en primer lugar se instalará una máquina virtual Java para ejecutar los servlets, por ejemplo las JVM de Sun o las de IBM, lo siguiente sería descargar e instalar un servidor como puede ser Tomcat.
Descargar Tomcat: https://tomcat.apache.org/
La instalación de Tomcat para Windows es muy simple, ya que con sólo ejecutar el instalador él detectará automáticamente J2SDK y configurará las variables de entorno. En Linux el proceso es algo más laborioso.

Instalación de Tomcat (para Linux):

1. ***Descargar el fichero zip/tar.gz/ desde: https://jakarta.apache.org/site/downloads/index.html***
2. ***Descomprimir el archivo en algún directorio, como por ejemplo: /home/usuario/tomcat***

3. *Definir las variables del entorno JAVA_HOME (con la ruta donde esté el JSDK, compilador Java) y*
   CATALINA_HOME (con la ruta del Tomcat). Para ello, se utiliza el comando:

```
export JAVA_HOME= “ruta del jsdk”
export CATALINA_HOME= “ruta del Tomcat”
```

4. *Para guardar correctamente las variables, teclear los comandos:*

```
echo $JAVA_HOME
echo $CATALINA_HOME
```

5. *Cerrar la consola.*

Un problema común en Linux es que los cambios de las variables de entorno no se guardan para la próxima vez que abra la consola. Esto se puede solucionar modificando el archivo “.bashrc”, se encuentra en el directorio personal de usuario, añadiéndole los comandos anteriores.

LA FORMA ANTERIOR NO ME HA FUNCIONADO PARA INSTALAR TOMCAT

```
sudo apt-get update
sudo apt-get -y install openjdk-17-jdk
```

A continuación, compruebe la instalación:

```
java -version
```

Deberá obtener un resultado similar al que se incluye a continuación:

```
$ java -version
openjdk version "17.0.3" 2022-04-19
OpenJDK Runtime Environment (build 17.0.3+7-Ubuntu-0ubuntu0.22.04.1)
OpenJDK 64-Bit Server VM (build 17.0.3+7-Ubuntu-0ubuntu0.22.04.1, modo mixto,
compartido)
```

Una práctica recomendable es definir una variable de entorno JAVA_HOME con la ruta de instalación de JDK.

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64/
echo $JAVA_HOME
```

Crea un nuevo grupo de tomcat que ejecutará el servicio:

```
sudo groupadd tomcat
```

Ahora, el siguiente paso es crear un nuevo usuario de tomcat. Agrega usuarios al grupo Tomcat con el directorio de inicio opt/tomcat para ejecutar el servicio Tomcat:

```
sudo useradd -s /bin/false -g tomcat -d /opt/tomcat tomcat
```

Instalar Tomcat en Ubuntu

```
cd /tmp
```

Para descargar desde el enlace copiado (desde el sitio web de Tomcat), usa el siguiente [comando curl](#):

```
curl -O https://www-us.apache.org/dist/tomcat/tomcat-9/v9.0.17/bin/apache-tomcat-9.0.17.tar.gz
```

O desde la web https://tomcat.apache.org/download-90.cgi descargar la version tar.gz de core

Paso 4: Permisos de actualización

Ahora que has terminado la instalación de Tomcat en Ubuntu, debes configurar el usuario de Tomcat para que tenga acceso completo a la instalación de Tomcat. Este usuario debe tener acceso al directorio. Usa los siguientes comandos:

```
sudo mkdir /opt/tomcat
cd /opt/tomcat
sudo tar xzvf /tmp/apache-tomcat-9.0.*tar.gz -C /opt/tomcat --strip-components=1
```

Ahora, otorga al grupo Tomcat la propiedad sobre todo el directorio de instalación con el [comando chgrp](#):

```
sudo chgrp -R tomcat /opt/tomcat
```

A continuación, debes otorgarle al usuario Tomcat acceso al directorio conf para ver su contenido y ejecutar el acceso al directorio en sí:

```
sudo chmod -R g+r conf
sudo chmod g+x conf
```

Convierte al usuario de Tomcat en el propietario de los directorios de apps, work, temp y logs:

```
sudo chown -R tomcat webapps/work/temp/logs/
```

Paso 5: Crear un archivo de unidad systemd

Necesitarás crear un nuevo archivo de unidad para ejecutar Tomcat como servicio. Abre tu editor de texto y crea un nombre de archivo tomcat.service en /etc/systemd/system/:

```
sudo nano /etc/systemd/system/tomcat.service
```

A continuación, copia y pega la siguiente configuración:

```
[Unit]
Description=Apache Tomcat Web Application Container
After=network.target
```

```
[Service]
Type=forking

Environment=JAVA_HOME=/usr/lib/jvm/java-1.8.0-openjdk-amd64/jre
Environment=CATALINA_PID=/opt/tomcat/temp/tomcat.pid
Environment=CATALINA_Home=/opt/tomcat
Environment=CATALINA_BASE=/opt/tomcat
Environment='CATALINA_OPTS=-Xms512M -Xmx1024M -server -XX:+UseParallelGC'
Environment='JAVA_OPTS.awt.headless=true -Djava.security.egd=file:/dev/v/urandom'

ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh

User=tomcat
Group=tomcat
UMask=0007
RestartSec=10
Restart=always

[Install]

WantedBy=multi-user.target
```

Guarda y cierra el archivo después de finalizar con los comandos anteriores.

A continuación, notifica al sistema que has creado un nuevo archivo emitiendo el siguiente comando en la línea de comando:

```
sudo systemctl daemon-reload
```

Los siguientes comandos te permitirán ejecutar el servicio Tomcat:

```
cd /opt/tomcat/bin
sudo ./startup.sh run
```

Paso 6: Ajustar el Firewall

Es esencial ajustar el firewall para que las solicitudes lleguen al servicio. Tomcat utiliza el puerto 8080 para aceptar solicitudes convencionales. Permite el tráfico a ese puerto utilizando UFW:

```
sudo ufw allow 8080
```

Para acceder a la página de inicio ve a tu dominio o dirección IP seguido de: 8080 en un navegador web – http://IP:8080

Configurar variable:

```
export CATALINA_HOME=/opt/tomcat/
```

La documentación de instalación de mi ordenador, sacada de : https://www.hostinger.es/tutoriales/como-instalar-tomcat-en-ubuntu

Una vez se han establecido las variables de entorno, se puede iniciar el servidor de la siguiente manera:
En Windows: ejecutar el archivo "startup.bat". En Linux: ejecutar el archivo "startup.sh" .
El servidor Tomcat se ejecuta en el puerto 8080.¡
Para detener el servidor hay que ejecutar ejecutar el archivo shutdown.
Para comprobar que el servidor funciona correctamente, en el navegador, la URL http://localhost:8080
Una vez comprobado que el servidor funciona correctamente, ya se está en disposición de crear una página JSP, utilizando cualquier editor de texto.

El código siguiente repite sucesivas veces la frase Hola mundo.

```
<!-- (c) JJ -->
<%@ page language='java' contentType="text/html" %>
<%! int count=0; %>
<html>
<head>`<title> Hola mundo. Intro to JSP`</title>`</head>
<body bgcolor="white">
Hola, mundo. Lo repito `<%= count++ %>
<% if (count == 1) { %>
Vez
 <% } else { %>
 Veces
 <% } %>
 </body>
</html>
```

Dicho Código tiene que estar almacenado en la ruta de instalacion de tomcat, en la carpeta webapps. ejemplo:

```
/opt/tomcat/webapps/examples/jsp
```

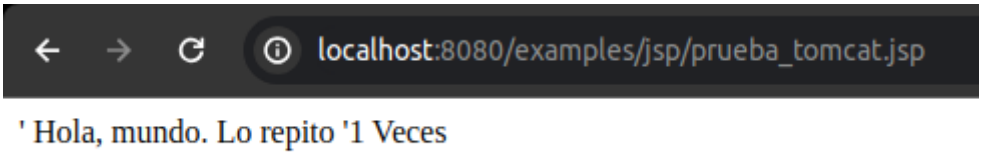
y desde el navegador:

```
http://localhost:8080/examples/jsp/prueba_tomcat.jsp
```

Para que funcione, hay que tener levantado el servicio de tomcat

```
cd /opt/tomcat/bin
./startup.sh
```

y aparecerá algo como:



' Hola, mundo. Lo repito '1 Veces

Para apagar el servicio de tomcat:

```
cd /opt/tomcat/bin
./shutdown.sh
```